

Univbersity of Catania
Degree course in Data Science

Supervised Analysis
Water Potability

Prepared by:

Andrea Isaia

Supervisor
Tomarchio Salvatore Daniele

2026

Contents

1 Introduction **2**

2 Description fo the dataset **2**

3 Missing values **3**

4 Univariate analysis **5**
4.1 Potable or Not Potable for each variable 5

5 Multivariate analysis **15**

6 Logistic Regression **17**
6.1 Univariate Logistic Regression 17
6.1.1 ph 17
6.1.2 Hardness 18
6.1.3 Solids 20
6.1.4 Chloramines 21
6.1.5 Sulfate 23
6.1.6 Conductivity 24
6.1.7 Organic_carbon 26
6.1.8 Trihalomethanes 27
6.1.9 Turbidity 29
6.2 Multivariate Logistic regression 30

7 LDA **32**

8 QDA **32**

9 Tree-based method **33**

10 Bagging, random forest and boosting **34**
10.1 Bagging 35
10.2 Random forest 35
10.3 Boosting 37

11 Conclusion **38**

12 Appendix A **39**

13 Appendix B **40**
13.1 Libraries 40
13.2 NA values 40
13.3 Unaviarate analysis 40
13.4 Logistic regression 42
13.5 Multivariate logistic regression 44

13.6 LDA 46
13.7 QDA 46
13.8 Tree-method approach 46
13.9 Bagging, rendom forest and boosting 47

1 Introduction

Our analysis is on water potability dataset. Our scope is to classify each registered water as potable or not depending on its chemical properties. The datasate can be downloaded from this site: https://www.kaggle.com/datasets/uom190346a/water-quality-and-potability.

2 Description fo the dataset

We first analyse our dataset.

```
'data.frame':   3276 obs. of  10 variables:
 $ ph              : num  NA 3.72 8.1 8.32 9.09 ...
 $ Hardness        : num  205 129 224 214 181 ...
 $ Solids          : num  20791 18630 19910 22018 17979 ...
 $ Chloramines     : num  7.3 6.64 9.28 8.06 6.55 ...
 $ Sulfate         : num  369 NA NA 357 310 ...
 $ Conductivity    : num  564 593 419 363 398 ...
 $ Organic_carbon  : num  10.4 15.2 16.9 18.4 11.6 ...
 $ Trihalomethanes: num  87 56.3 66.4 100.3 32 ...
 $ Turbidity       : num  2.96 4.5 3.06 4.63 4.08 ...
 $ Potability      : int  0 0 0 0 0 0 0 0 0 0 ...
```

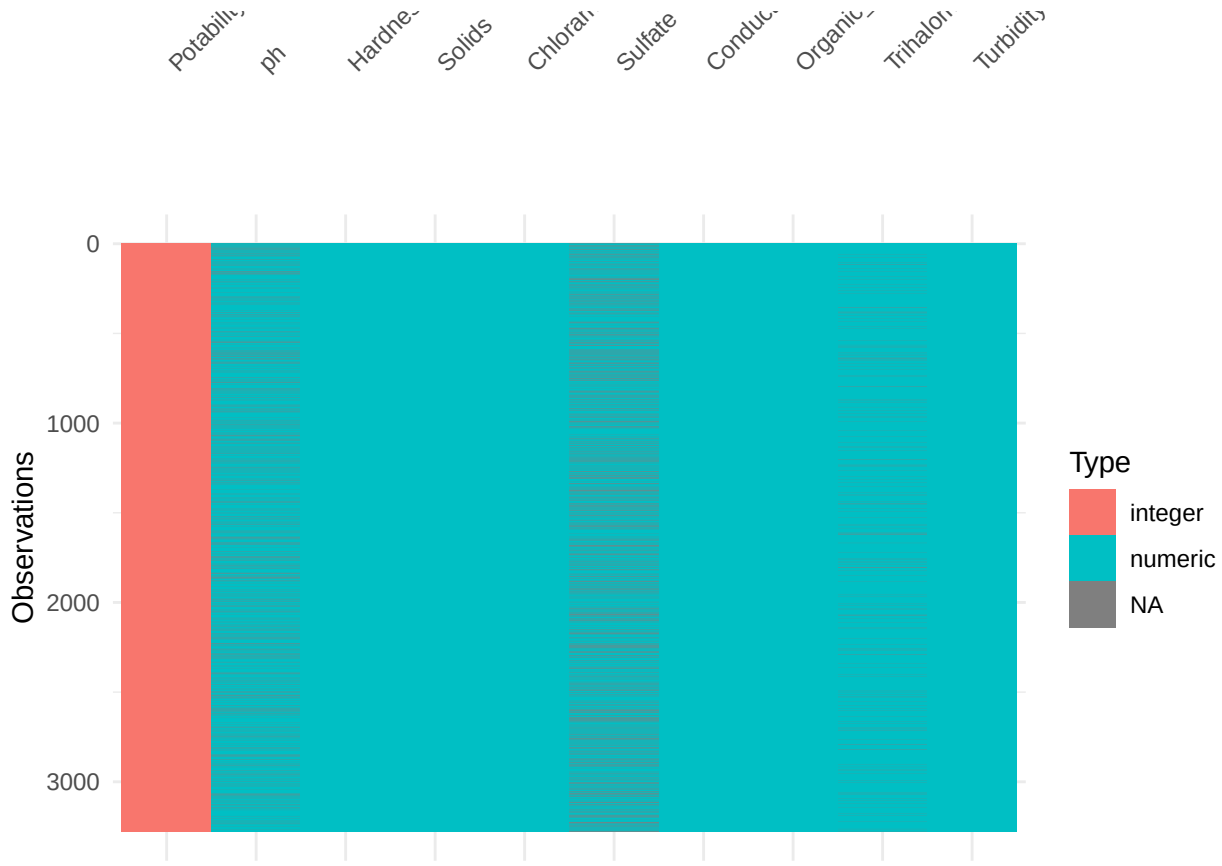
There are 10 variables. In particular:

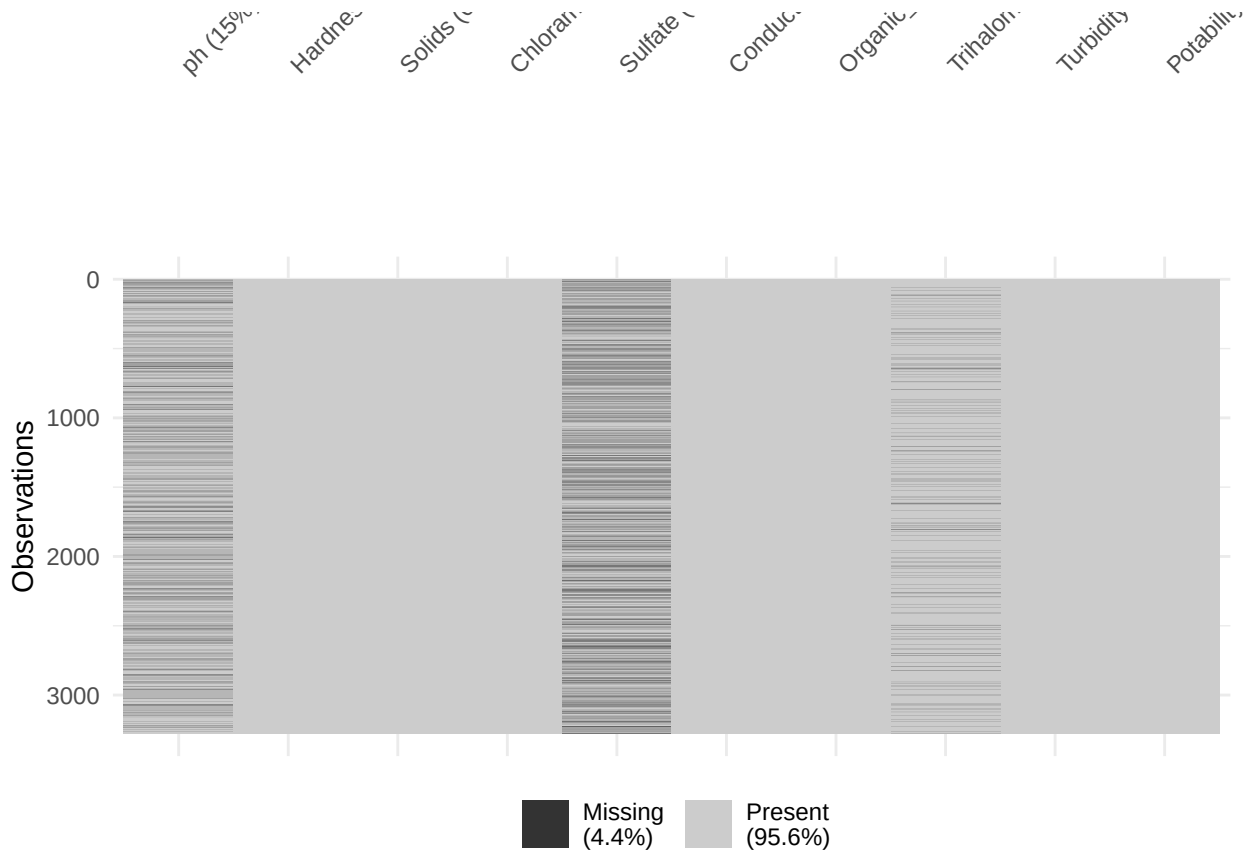
- 9 variables are represented by real number:
  - **ph**: The ph level fo the water.
  - **Hardnes**: A measure of mineral content.
  - **Solids**: Total dissolved solids in the water.
  - **Chloramines**: Chloramines concentration in the water.
  - **Sulfate**: Sulfate concentration in the water.
  - **Conductivity**: Electrical conductivity of the water.
  - **Organic carbon**: Organic carbon content in the water.
  - **Trihalomethanes**: Trihalomethanes concentration in the water.
  - **Turbidity**: Turbidity level, a measure of water clarity.
- 1 integer variable **Potability** that can be 0 or 1 to represent respectively not potable and potable.

3 Missing values

Now we analyse missing values in our dataset.

```
# A tibble: 10 x 3
   variable         n_miss pct_miss
   <chr>             <int>    <num>
 1 Sulfate             781    23.8
 2 ph                  491    15.0
 3 Trihalomethanes     162     4.95
 4 Hardness              0     0
 5 Solids                0     0
 6 Chloramines           0     0
 7 Conductivity          0     0
 8 Organic_carbon        0     0
 9 Turbidity             0     0
10 Potability            0     0
```





From the graphs we can notice that only three of the original variables are missing. The first table shows the number of missing values and its percentage. The second and the third graph shows which value is missing. Note that the 4.4% refers to the total number of missing values across all measurements for all variables.

To better understand the distribution of the data across each class, we analyse only NA for each variable depending on the potability.

```
Percentage of missing values for ph in Not Potable water:  24.42442
Percentage of missing values for ph in Potable water:  22.92645
 ----------------------------------------
Percentage of missing values for ph in Not Potable water:  15.71572
Percentage of missing values for ph in Potable water:  13.84977
 ----------------------------------------
Percentage of missing values for ph in Not Potable water:  5.355355
Percentage of missing values for ph in Potable water:  4.303599
 ----------------------------------------
```

The percentage is calculated over the number of rows related to each class. The result highlights the possibility of MCAR.

We need to understand the number of rows that have at least one NA value.

```
Total number of rows: 3276
```

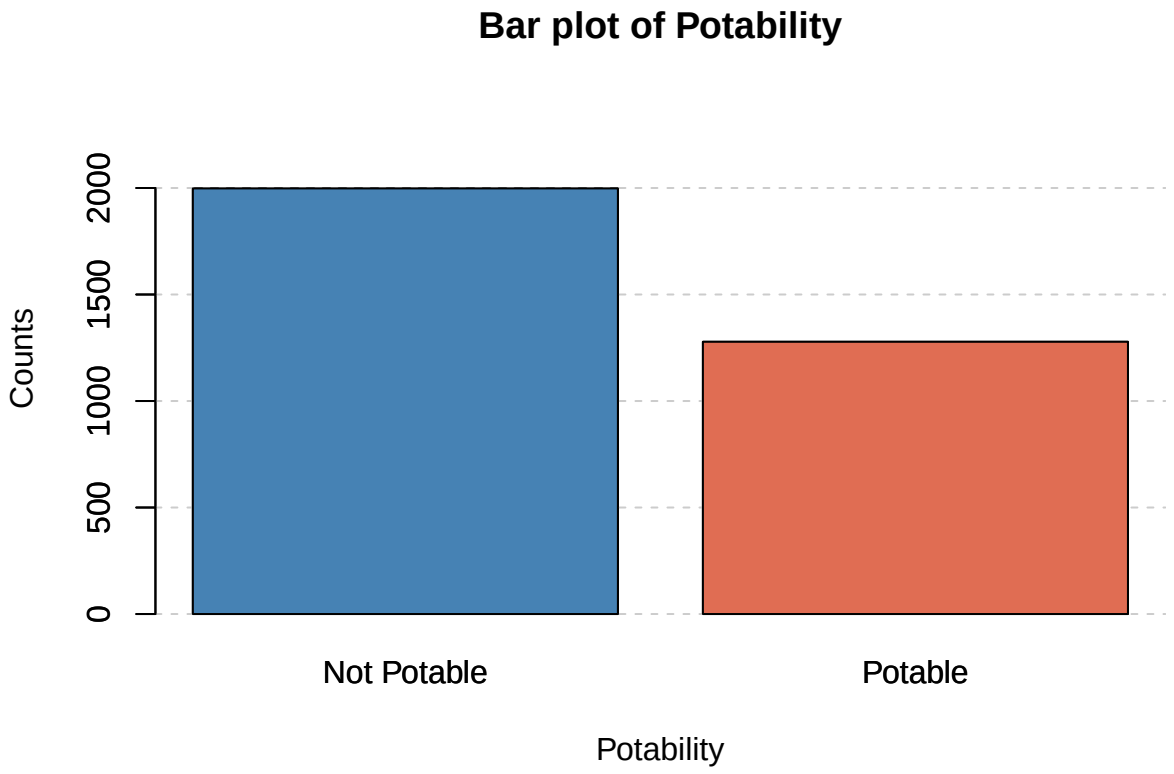
```
Remaining number of rows without NA: 2011
Number of rows with NA: 1265
Percentage of rows with NA values: 38.61 %
```

We cannot delete all these rows from our analysis, it would affect our statistical result. So I decided to substitute to NA value the respective median for that variable.

4 Univariate analysis

First of all we analyse our potability variable.

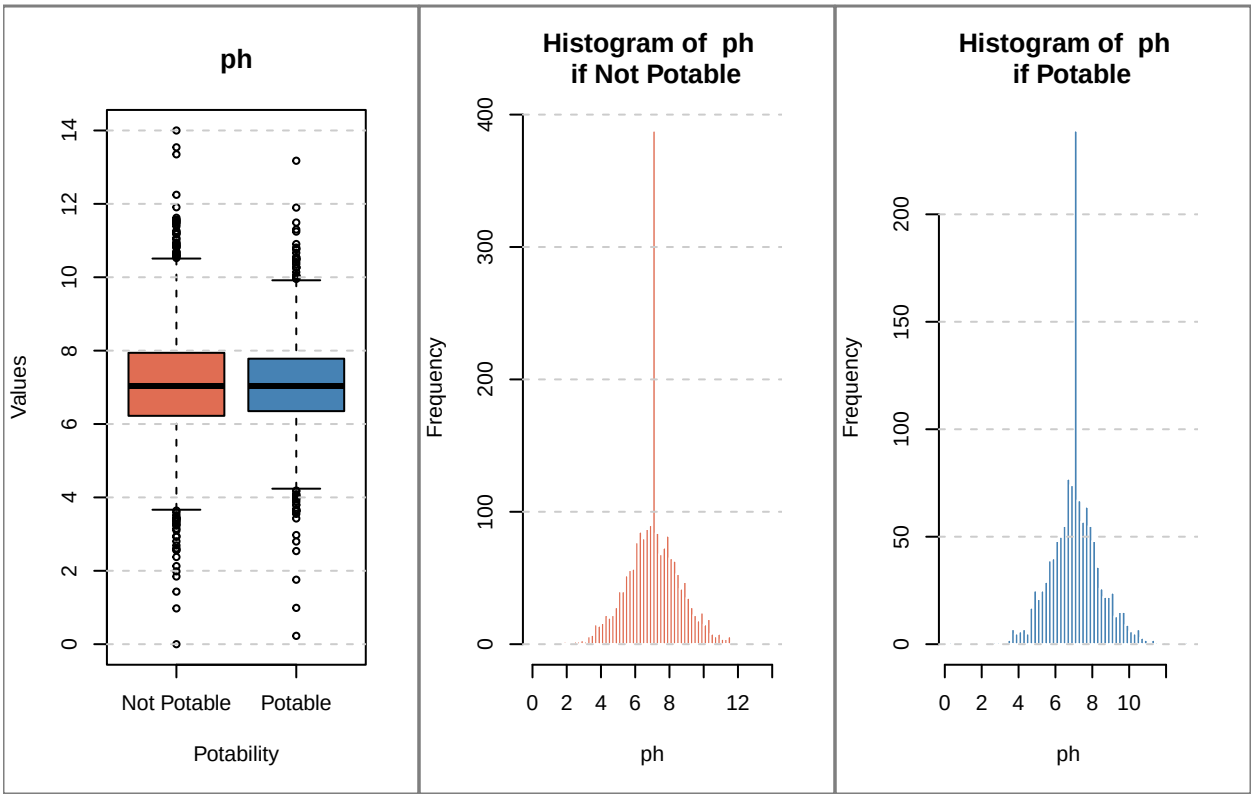
```
     Category Frequency Relative_frequecy
1 Not Potable      1998             60.99%
2     Potable      1278             39.01%
```



4.1 Potable or Not Potable for each variable

Now we will analyse the incidence of potability for each variable. It's represented by the boxplot comparing Potable case vs Not Potable case and then adding histograms of these two cases and their summary.

ph



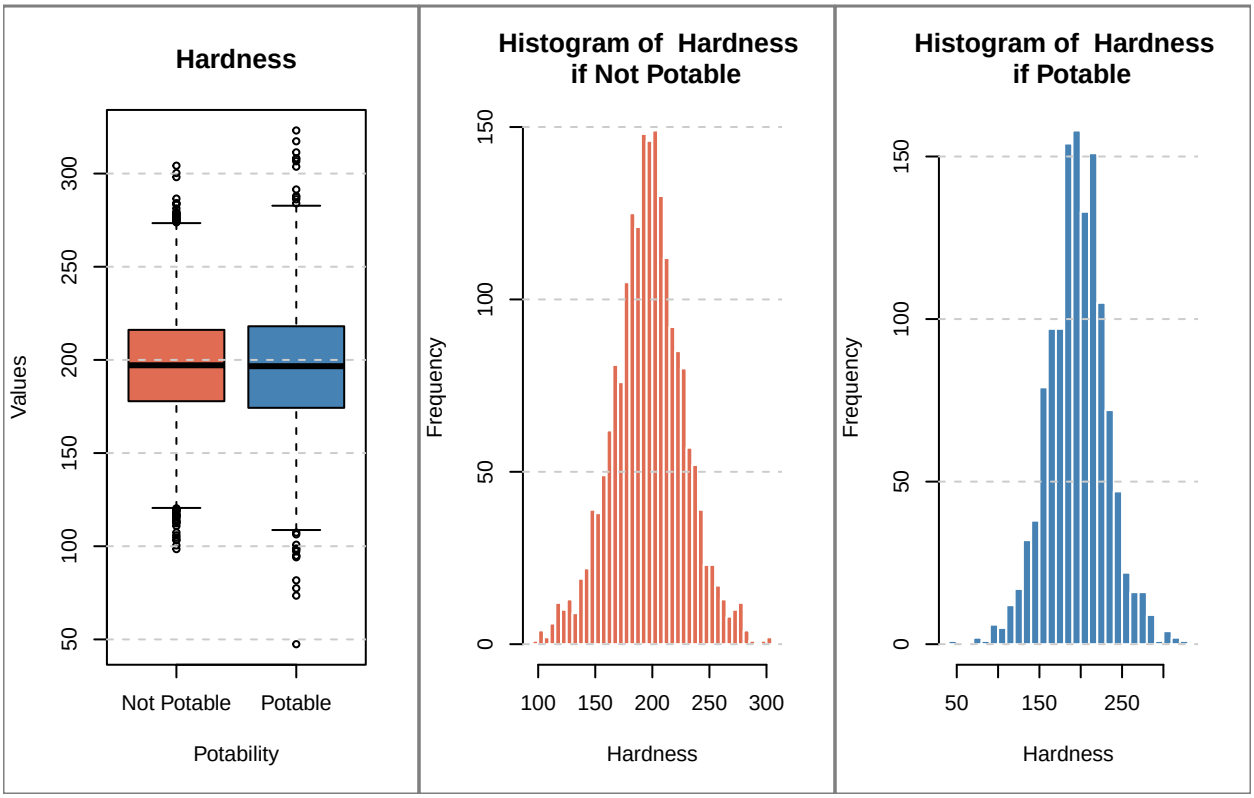

```
Summary of  ph  if Not Potable

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  0.000   6.224   7.037   7.078   7.941  14.000
Variance :  2.388849
Standard deviation :  1.54559
Mode :  7.03675210383355
Kurtosis :  4.161606
Skewness :  0.05992251


Summary of  ph  if Potable

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 0.2275  6.3518  7.0368  7.0687  7.7801 13.1754
Variance :  1.806372
Standard deviation :  1.344013
Mode :  7.03675210383355
Kurtosis :  4.667341
Skewness :  -0.008072517
```

Hardness



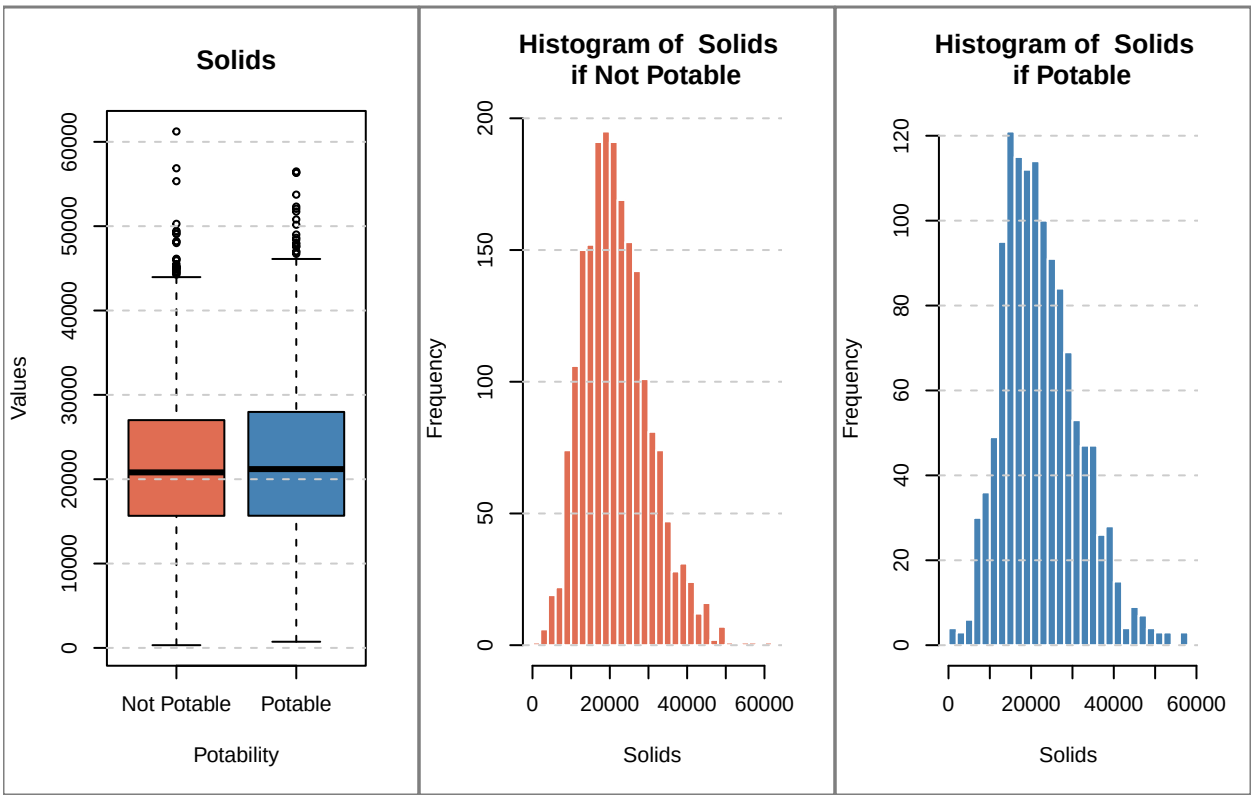

```
Summary of  Hardness  if Not Potable

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  98.45  177.82  197.12  196.73  216.12  304.24
Variance :  964.5708
Standard deviation :  31.05754
Mode :  98.4529305095863
Kurtosis :  3.360482
Skewness :  -0.02147507


Summary of  Hardness  if Potable

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  47.43  174.33  196.63  195.80  218.00  323.12
Variance :  1263.592
Standard deviation :  35.54704
Mode :  47.432
Kurtosis :  3.718281
Skewness :  -0.04607102
```

Solids



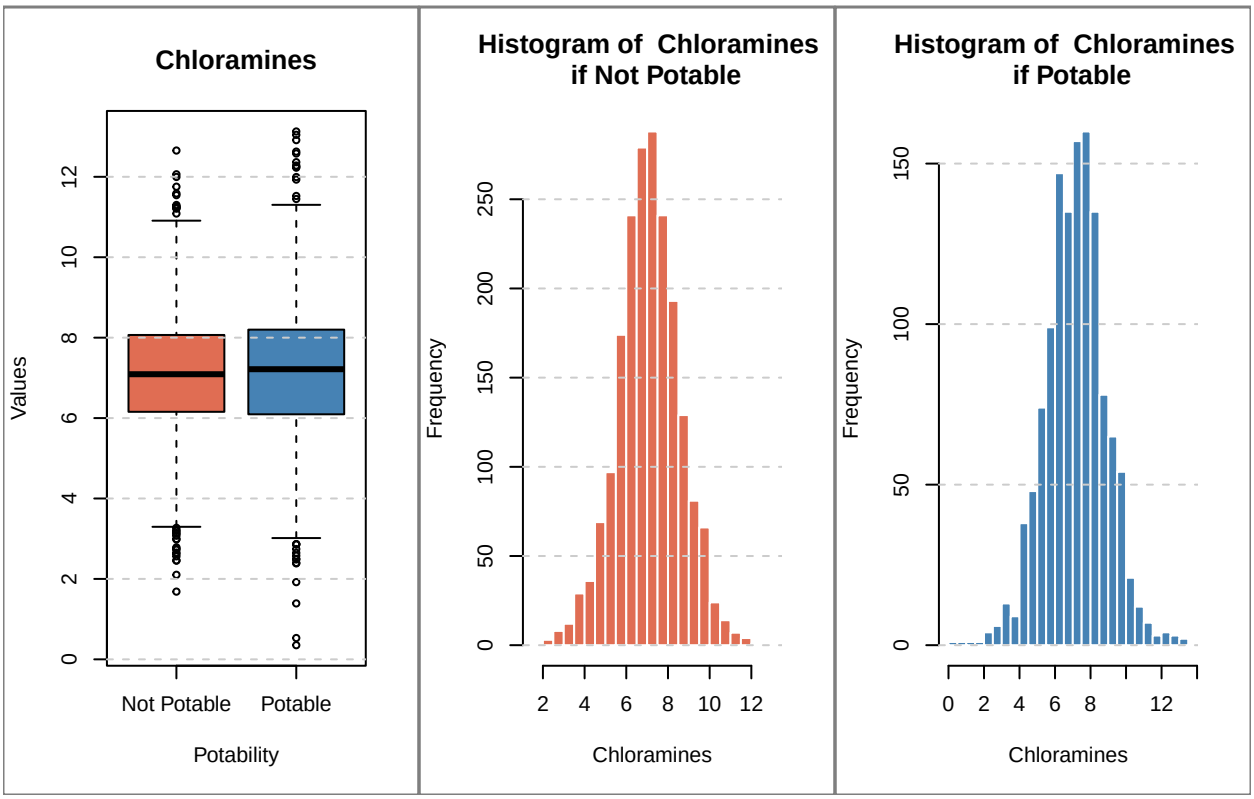

```
Summary of  Solids  if Not Potable

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  320.9 15663.1 20809.6 21777.5 27006.2 61227.2
Variance :  72984024
Standard deviation :  8543.069
Mode :  320.942611274359
Kurtosis :  3.442207
Skewness :  0.6057294


Summary of  Solids  if Potable

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  728.8 15669.0 21199.4 22384.0 27973.2 56488.7
Variance :  82828387
Standard deviation :  9101.01
Mode :  728.750829576846
Kurtosis :  3.389786
Skewness :  0.6274181
```

Chloramines



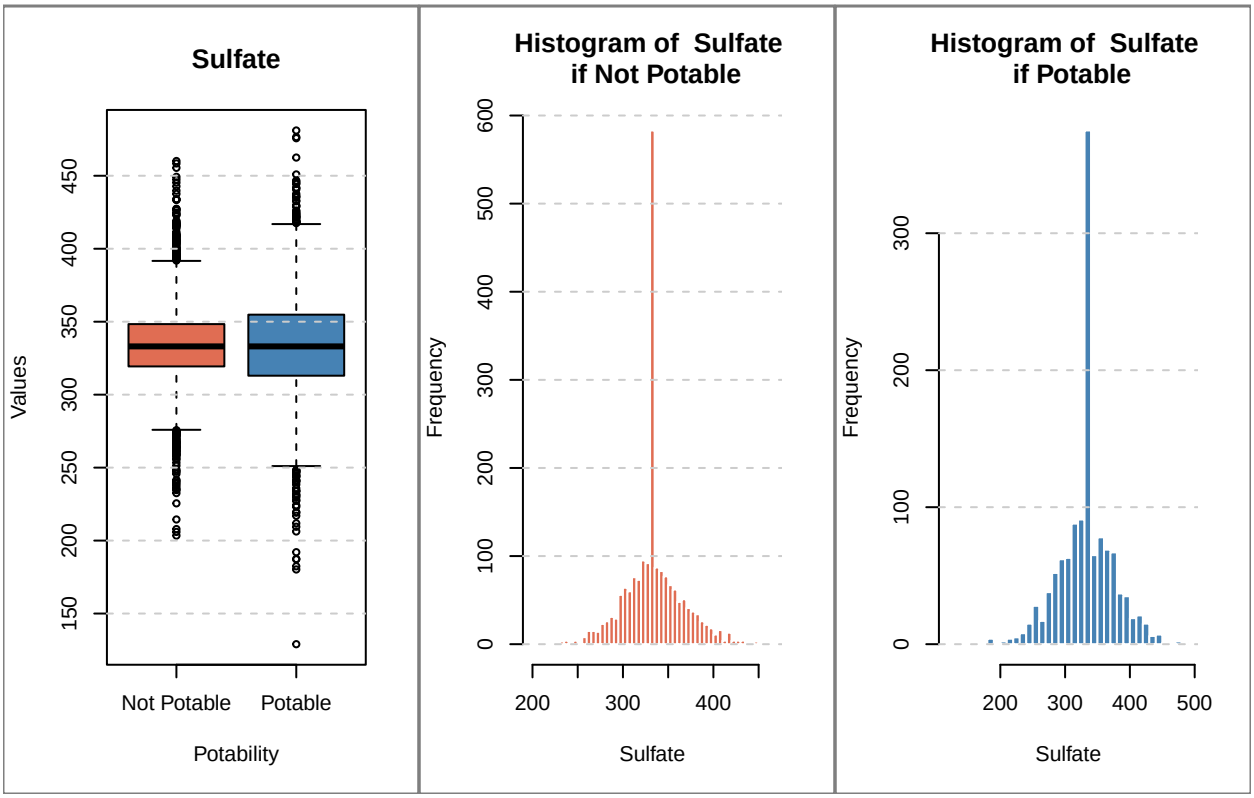

```
Summary of  Chloramines  if Not Potable

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  1.684   6.156   7.090   7.092   8.066  12.653
Variance :  2.253136
Standard deviation :  1.501045
Mode :  1.683992580728
Kurtosis :  3.362795
Skewness :  -0.02227379


Summary of  Chloramines  if Potable

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  0.352   6.094   7.215   7.169   8.199  13.127
Variance :  2.900167
Standard deviation :  1.702988
Mode :  0.352
Kurtosis :  3.691713
Skewness :  -0.01956101
```

Sulfate



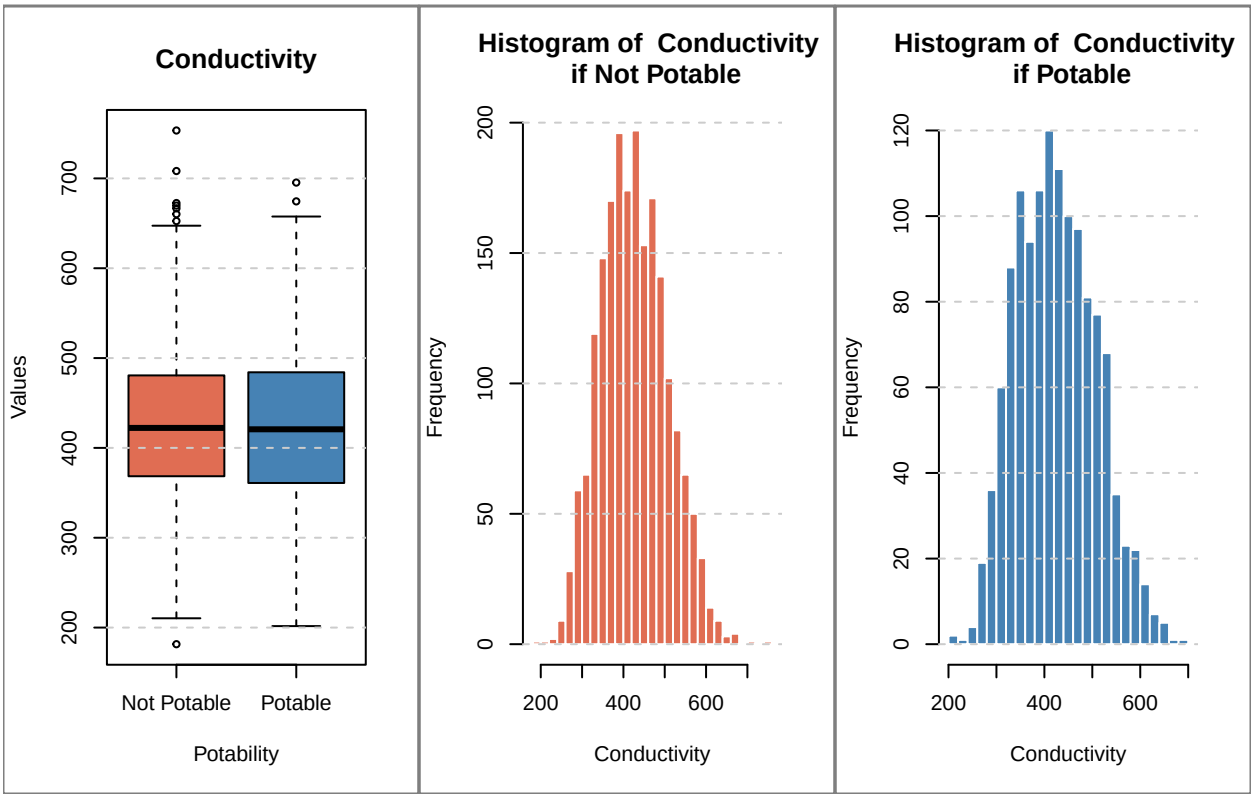

```
Summary of  Sulfate  if Not Potable

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  203.4   319.4   333.1   334.2   348.3   460.1
Variance :  1020.693
Standard deviation :  31.94829
Mode :  333.073545745888
Kurtosis :  4.603708
Skewness :  0.1700779


Summary of  Sulfate  if Potable

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  129.0   313.1   333.1   332.7   354.8   481.0
Variance :  1752.756
Standard deviation :  41.86593
Mode :  333.073545745888
Kurtosis :  4.354246
Skewness :  -0.1355178
```

Conductivity



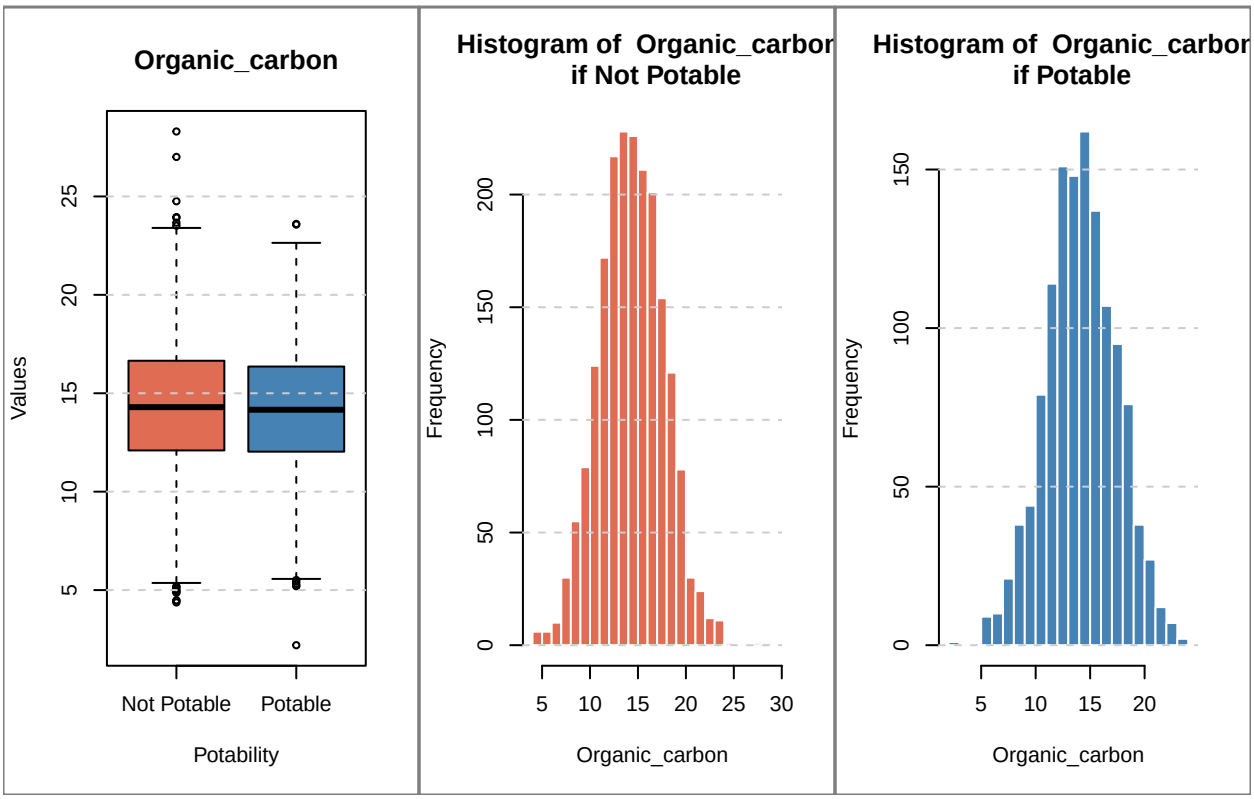

```
Summary of  Conductivity  if Not Potable

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  181.5   368.5   422.2   426.7   480.7   753.3
Variance :  6407.573
Standard deviation :  80.04732
Mode :  181.483753985146
Kurtosis :  2.802537
Skewness :  0.2582372


Summary of  Conductivity  if Potable

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  201.6   360.9   420.7   425.4   484.2   695.4
Variance :  6731.948
Standard deviation :  82.04845
Mode :  201.619736755158
Kurtosis :  2.604137
Skewness :  0.2746385
```

Organic_carbon



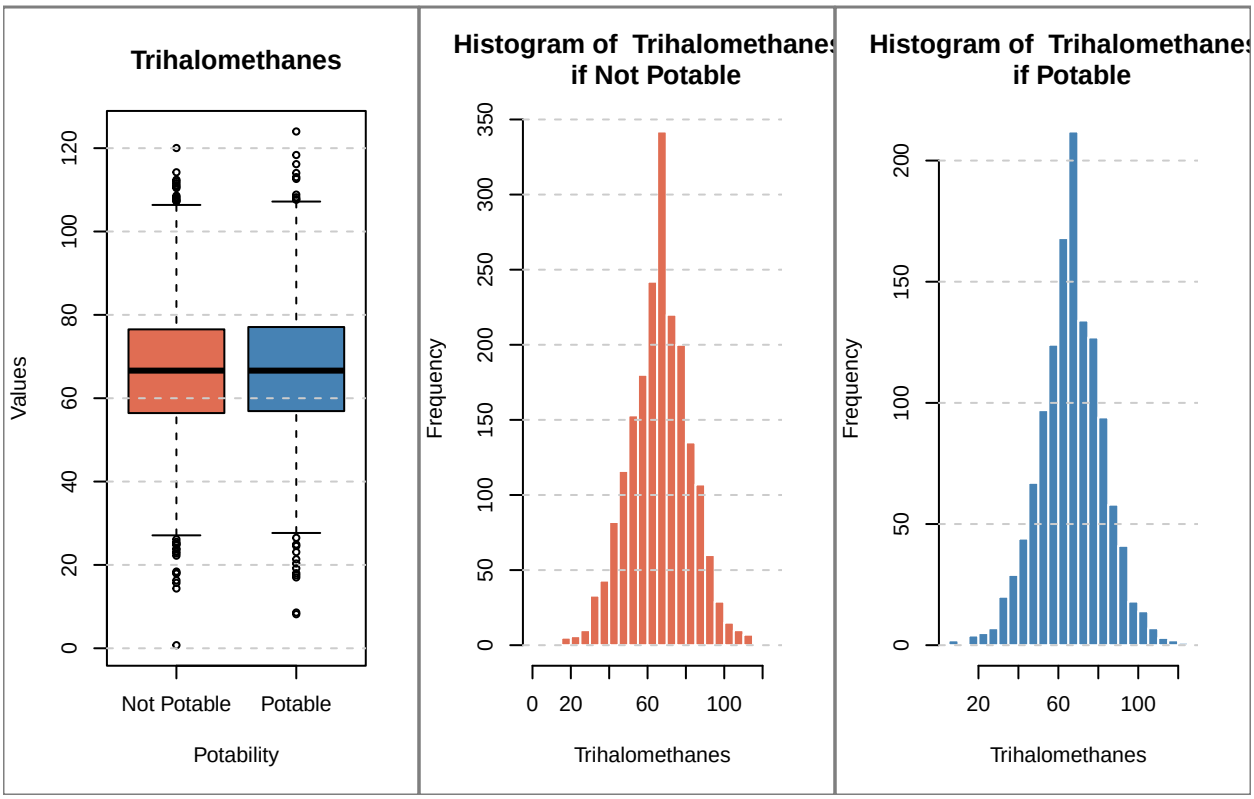

```
Summary of  Organic_carbon  if Not Potable

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  4.372  12.101  14.294  14.364  16.649  28.300
Variance :  11.11925
Standard deviation :  3.334554
Mode :  4.37189860844114
Kurtosis :  3.085005
Skewness :  0.06888924


Summary of  Organic_carbon  if Potable

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   2.20   12.03   14.16   14.16   16.36   23.60
Variance :  10.65309
Standard deviation :  3.263907
Mode :  2.19999999999999
Kurtosis :  2.947793
Skewness :  -0.05172222
```

Trihalomethanes



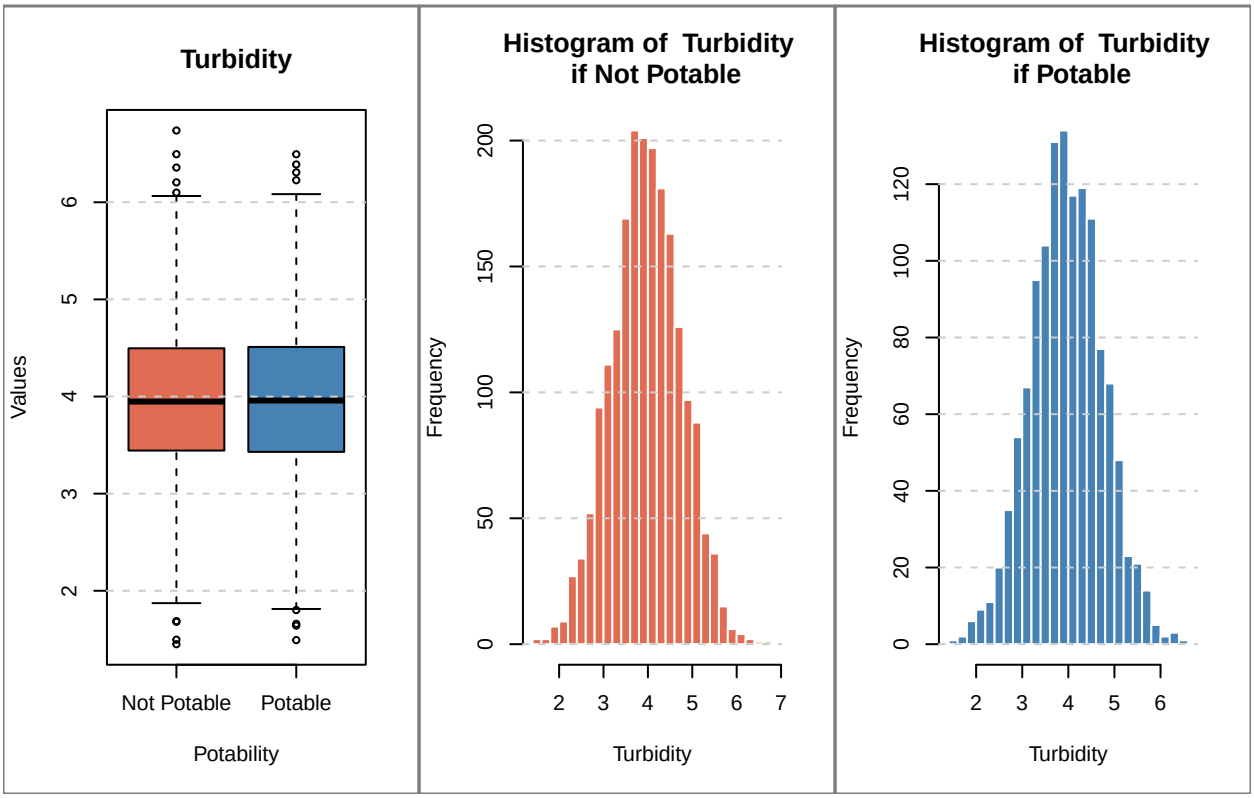

```
Summary of  Trihalomethanes  if Not Potable

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  0.738  56.442  66.622  66.321  76.519 120.030
Variance :  244.6968
Standard deviation :  15.64279
Mode :  66.6224850980848
Kurtosis :  3.325951
Skewness :  -0.1025565


Summary of  Trihalomethanes  if Potable

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  8.176  56.911  66.622  66.543  77.067 124.000
Variance :  255.1032
Standard deviation :  15.97195
Mode :  66.6224850980848
Kurtosis :  3.51399
Skewness :  -0.06577326
```

Turbidity




```
Summary of  Turbidity  if Not Potable

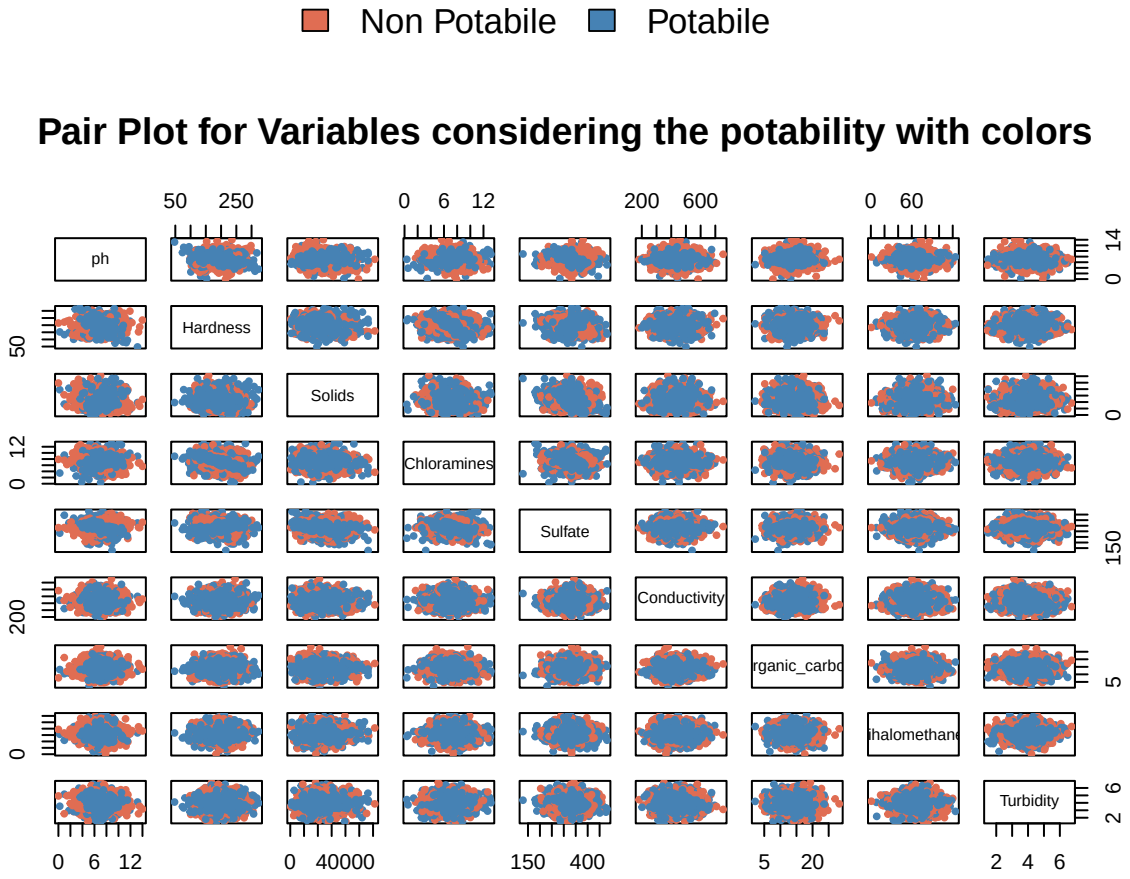
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  1.450   3.444   3.948   3.966   4.496   6.739
Variance :  0.6088406
Standard deviation :  0.7802824
Mode :  1.45
Kurtosis :  2.888511
Skewness :  -0.009191307


Summary of  Turbidity  if Potable

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  1.492   3.431   3.959   3.968   4.510   6.494
Variance :  0.6097139
Standard deviation :  0.7808418
Mode :  1.49220661530425
Kurtosis :  3.00867
Skewness :  -0.005668957
```

5 Multivariate analysis

Now we will use the multivariate analysis to better understand the link between variables. First of all we plot all with the pair plot.



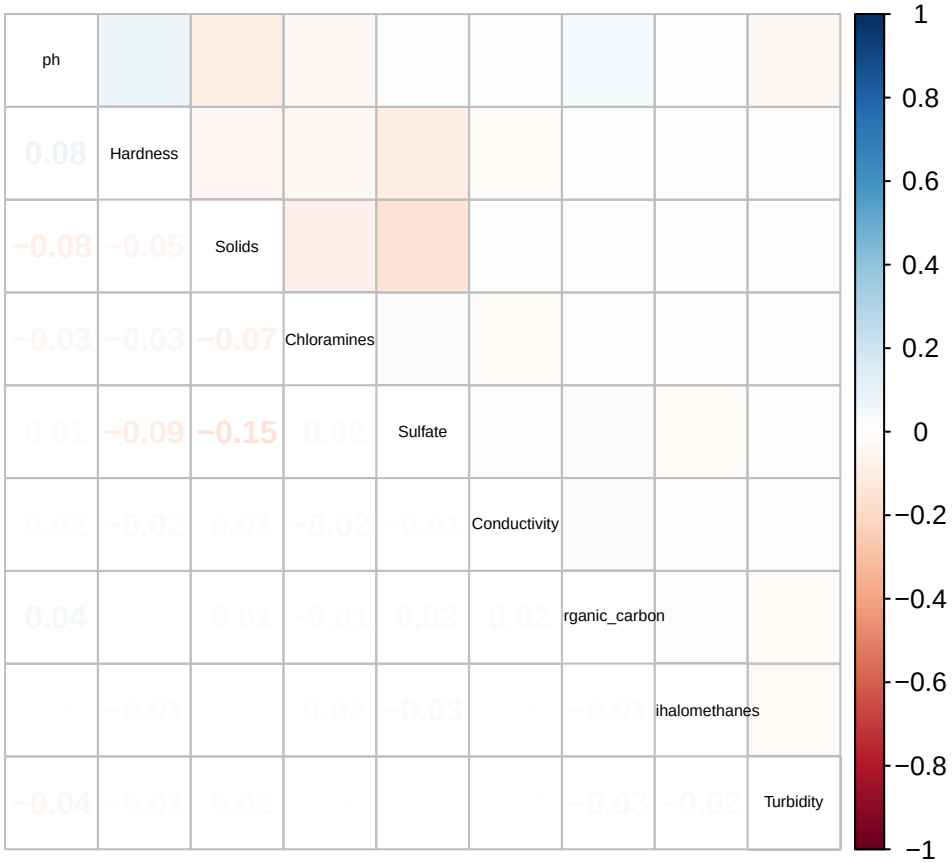
In general we can observe an ellipsoid distribution for each pair and on the border of it not potable variables. This means that a combination of high or low level of two variables is sufficient to underline a not potable water.

```
                          ph     Hardness        Solids  Chloramines
    Sulfate
ph                1.000000000  0.07576030 -0.082004177 -0.031740701
    0.014178441
Hardness          0.075760303  1.00000000 -0.046899365 -0.030054239
    -0.092833198
Solids           -0.082004177 -0.04689937  1.000000000 -0.070147580
    -0.149747347
Chloramines      -0.031740701 -0.03005424 -0.070147580  1.000000000
    0.023761688
Sulfate           0.014178441 -0.09283320 -0.149747347  0.023761688
    1.000000000
Conductivity      0.017465717 -0.02391460  0.013830898 -0.020486409
    -0.014182379
```

```
Organic_carbon    0.040240175  0.00361004  0.010242343 -0.012653471
   0.027102173
Trihalomethanes   0.003145084 -0.01270674 -0.008798626  0.016614101
   -0.025657186
Turbidity        -0.036106998 -0.01444913  0.019546144  0.002363321
   -0.009767387

                 Conductivity Organic_carbon Trihalomethanes    Turbidity
ph                0.017465717     0.04024018     0.003145084 -0.036106998
Hardness         -0.023914600     0.00361004    -0.012706744 -0.014449129
Solids            0.013830898     0.01024234    -0.008798626  0.019546144
Chloramines      -0.020486409    -0.01265347     0.016614101  0.002363321
Sulfate          -0.014182379     0.02710217    -0.025657186 -0.009767387
Conductivity      1.000000000     0.02096636     0.001184083  0.005798168
Organic_carbon    0.020966361     1.00000000    -0.012958281 -0.027308113
Trihalomethanes   0.001184083    -0.01295828     1.000000000 -0.021486733
Turbidity         0.005798168    -0.02730811    -0.021486733  1.000000000
```

All variables are not heavily correlated, meaning no collinearity. This will not affect our analysis.



Also this graph give us the evidence of no correlation: almost every pair of variables is colored white, indicating no correlation.

6 Logistic Regression

First, we start our analysis doing the univariate case of the logistic regression, then we will go through the multivariate case.

6.1 Univariate Logistic Regression

First we divide the dataset in Train and Test, respectively 70/30.

After the split we train our model on the training set and see the result for each variable. Then we do an analysis on polynomials of different grade up to 5 for each variable. This step is done using the CV aprroach to choose the best model among all of them.

```
             Potable
Not Potable        0
Potable            1
```

We can notice that for our model Potable is the probability that the logistic function will predict.

6.1.1 ph

```

Call:
glm(formula = Potability ~ ph, family = binomial, data = Water.L)

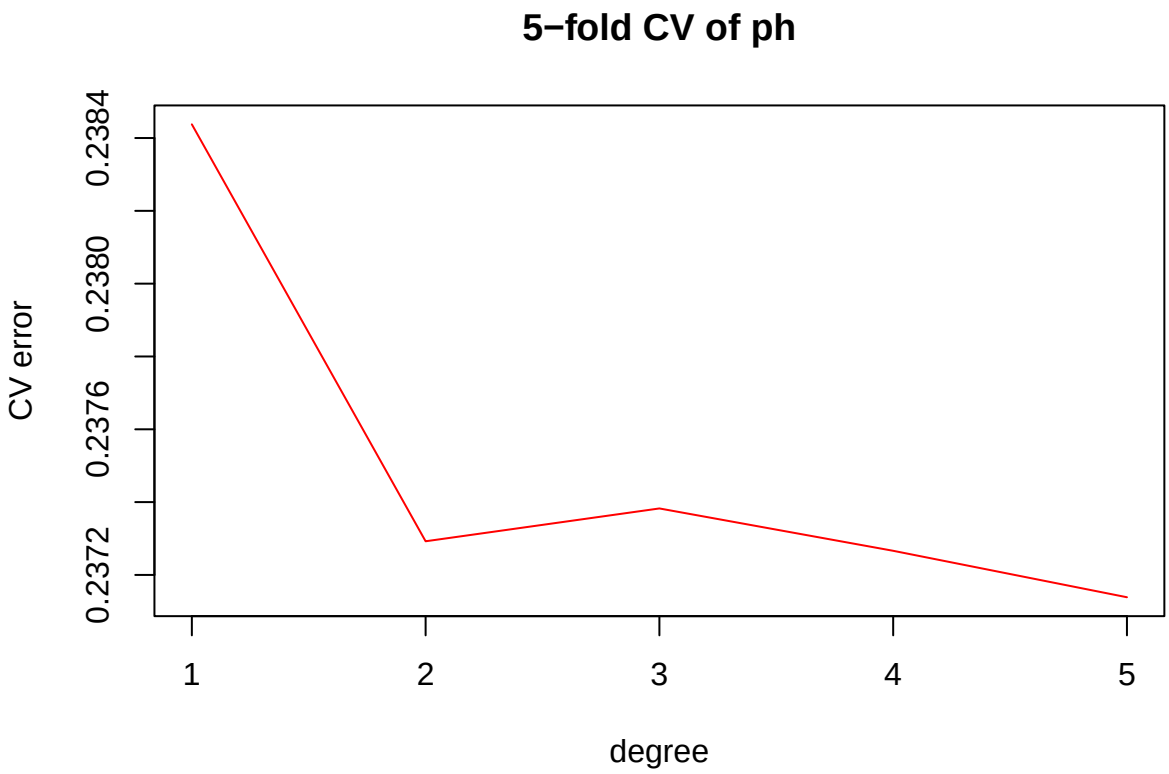
Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept) -0.494058   0.211375  -2.337   0.0194 *
ph           0.006713   0.029329   0.229   0.8190
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 3068.5  on 2293  degrees of freedom
Residual deviance: 3068.5  on 2292  degrees of freedom
AIC: 3072.5

Number of Fisher Scoring iterations: 4
```

In this case $Pr(>|z|) > 0.05$ for β_1, then we cannot reject the hypothesis that $\beta_1 = 0$. This means that there is no evidence that the potability depends **only** on that variable.



There is no significant change in the CV error. All of this means that any polynomial until degree 5 does not affect significantly the result.

```
Confusion matrix for training set of ph
glm.pred2.L.ph Not Potable Potable  Sum
   Not Potable        1399     895 2294
   Sum                1399     895 2294
Accuracy: 60.98518
Err: 39.01482

Confusion matrix for test set of ph
glm.pred2.T.ph Not Potable Potable Sum
   Not Potable         599     383 982
   Sum                 599     383 982
Accuracy: 60.99796
Err: 39.00204
```

The result confirm our hypothesis: the model cannot predict correctly our result.

6.1.2 Hardness

```

Call:
glm(formula = Potability ~ Hardness, family = binomial, data = Water.L)
```

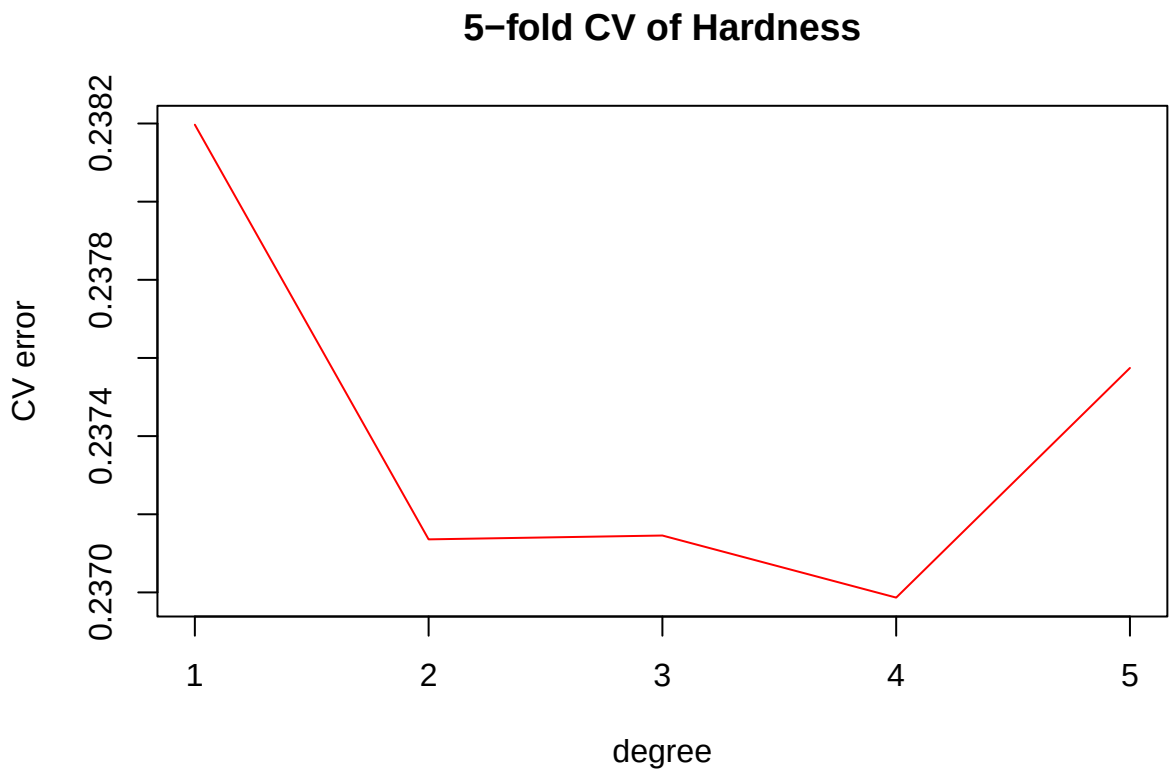
```
Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept) -0.249451   0.258540  -0.965    0.335
Hardness    -0.001006   0.001301  -0.773    0.439

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 3068.5  on 2293  degrees of freedom
Residual deviance: 3067.9  on 2292  degrees of freedom
AIC: 3071.9

Number of Fisher Scoring iterations: 4
```

In this case $Pr(>|z|) > 0.05$ for β_1, then we cannot reject the hypothesis that $\beta_1 = 0$. This means that there is no evidence that the potability depends **only** on that variable. We can notice that also $\beta_0 = 0$.



There is no significant change in the CV error. All of this means that any polynomial until degree 5 does not affect significantly the result.

```
Confusion matrix for training set of Hardness
glm.pred2.L.H Not Potable Potable  Sum
  Not Potable        1399     895 2294
```

```
  Sum                 1399      895 2294
Accuracy: 60.98518
Err: 39.01482

Confusion matrix for test set of Hardness
glm.pred2.T.H Not Potable Potable Sum
  Not Potable          599      383 982
  Sum                  599      383 982
Accuracy: 60.99796
Err: 39.00204
```

The result confirm our hypothesis: the model cannot predict correctly our result.

6.1.3 Solids

```

Call:
glm(formula = Potability ~ Solids, family = binomial, data = Water.L)

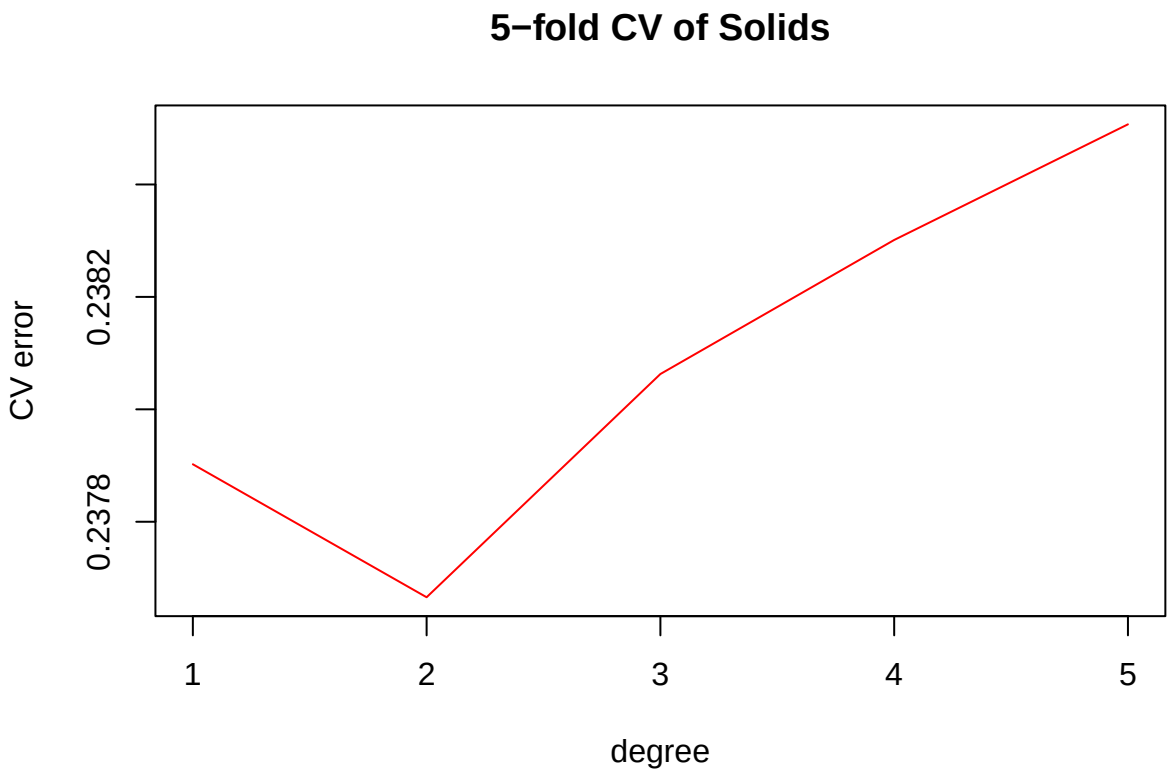
Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept) -6.627e-01  1.151e-01  -5.759 8.48e-09 ***
Solids       9.783e-06  4.820e-06   2.030   0.0424 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 3068.5  on 2293  degrees of freedom
Residual deviance: 3064.4  on 2292  degrees of freedom
AIC: 3068.4

Number of Fisher Scoring iterations: 4
```

Only on this variable we can reject the null hypothesis.



There is no significant change in the CV error, but also we have an increase of CV error. All of this means that any polynomial until degree 5 does not affect significantly the result.

```
Confusion matrix for training set of Solids
glm.pred2.L.S Not Potable Potable  Sum
  Not Potable        1399     895 2294
  Sum                1399     895 2294
Accuracy: 60.98518
Err: 39.01482

Confusion matrix for test set of Solids
glm.pred2.T.S Not Potable Potable Sum
  Not Potable         599     383 982
  Sum                 599     383 982
Accuracy: 60.99796
Err: 39.00204
```

In this case the model cannot predict correctly no one of our results. So it useless.

6.1.4 Chloramines

```

Call:
glm(formula = Potability ~ Chloramines, family = binomial, data = Water.L)
```

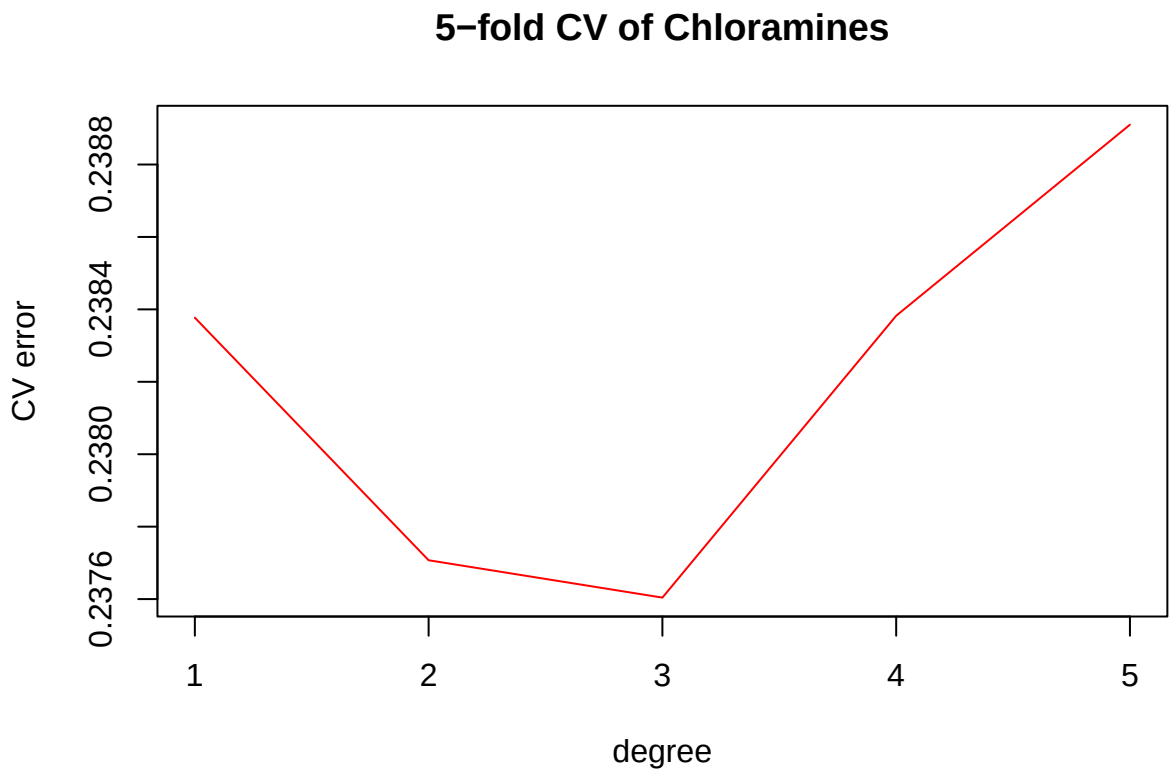
```
Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept) -0.425541   0.199029  -2.138   0.0325 *
Chloramines -0.002962   0.027229  -0.109   0.9134
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 3068.5  on 2293  degrees of freedom
Residual deviance: 3068.5  on 2292  degrees of freedom
AIC: 3072.5

Number of Fisher Scoring iterations: 4
```

In this case $Pr(>|z|) > 0.05$ for β_1, then we cannot reject the hypothesis that $\beta_1 = 0$. This means that there is no evidence that the potability depends **only** on that variable.



There is no significant change in the CV error. All of this means that any polynomial until degree 5 does not affect significantly the result.

```
Confusion matrix for training set of Chloramines
glm.pred2.L.C Not Potable Potable  Sum
  Not Potable        1399     895 2294
```

```
  Sum                 1399      895 2294
Accuracy: 60.98518
Err: 39.01482

Confusion matrix for test set of Chloramines
glm.pred2.T.C Not Potable Potable Sum
  Not Potable         599      383 982
  Sum                 599      383 982
Accuracy: 60.99796
Err: 39.00204
```

The result confirm our hypothesis: the model cannot predict correctly our result.

6.1.5 Sulfate

```

Call:
glm(formula = Potability ~ Sulfate, family = binomial, data = Water.L)

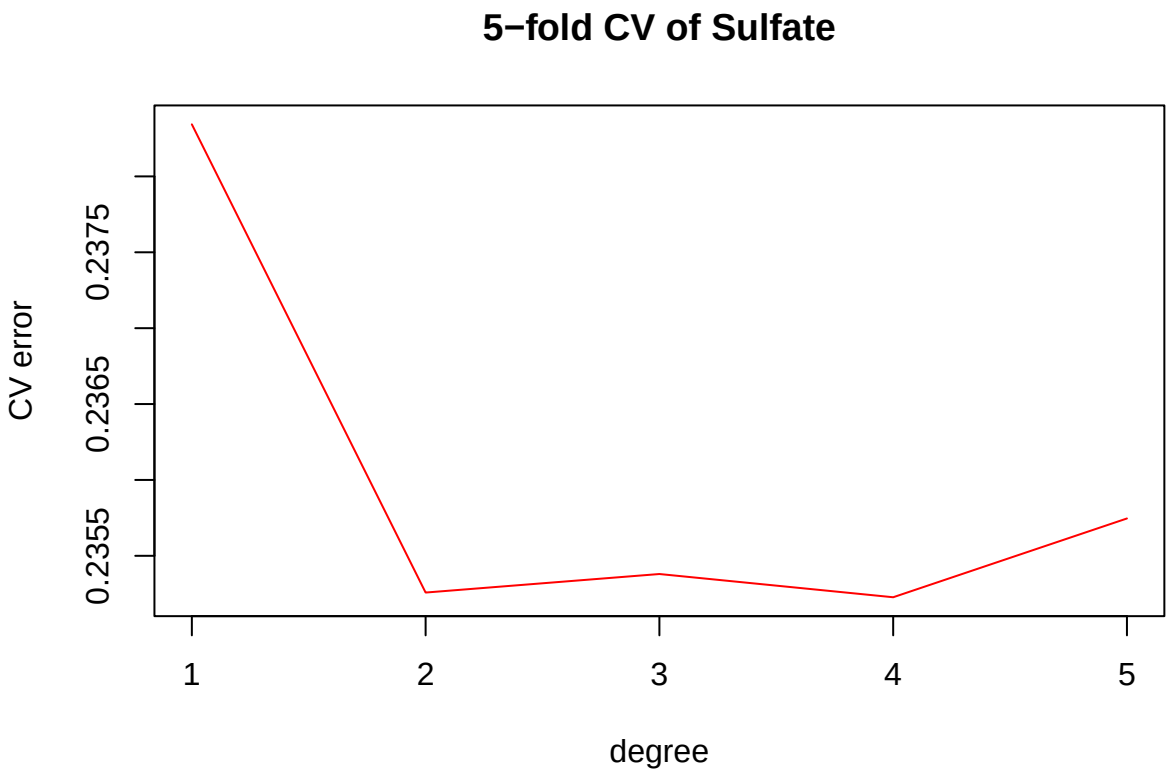
Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept) -0.020903   0.396260  -0.053    0.958
Sulfate     -0.001276   0.001181  -1.080    0.280

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 3068.5  on 2293  degrees of freedom
Residual deviance: 3067.4  on 2292  degrees of freedom
AIC: 3071.4

Number of Fisher Scoring iterations: 4
```

In this case $Pr(>|z|) > 0.05$ for β_1, then we cannot reject the hypothesis that $\beta_1 = 0$. This means that there is no evidence that the potability depends **only** on that variable.



There is no significant change in the CV error. All of this means that any polynomial until degree 5 does not affect significantly the result.

```
Confusion matrix for training set of Sulfate
glm.pred2.L.Su Not Potable Potable  Sum
   Not Potable        1399     895 2294
   Sum                1399     895 2294
Accuracy: 60.98518
Err: 39.01482

Confusion matrix for test set of Sulfate
glm.pred2.T.Su Not Potable Potable Sum
   Not Potable         599     383 982
   Sum                 599     383 982
Accuracy: 60.99796
Err: 39.00204
```

The result confirm our hypothesis: the model cannot predict correctly our result.

6.1.6 Conductivity

```

Call:
glm(formula = Potability ~ Conductivity, family = binomial, data = Water.L
    )
```

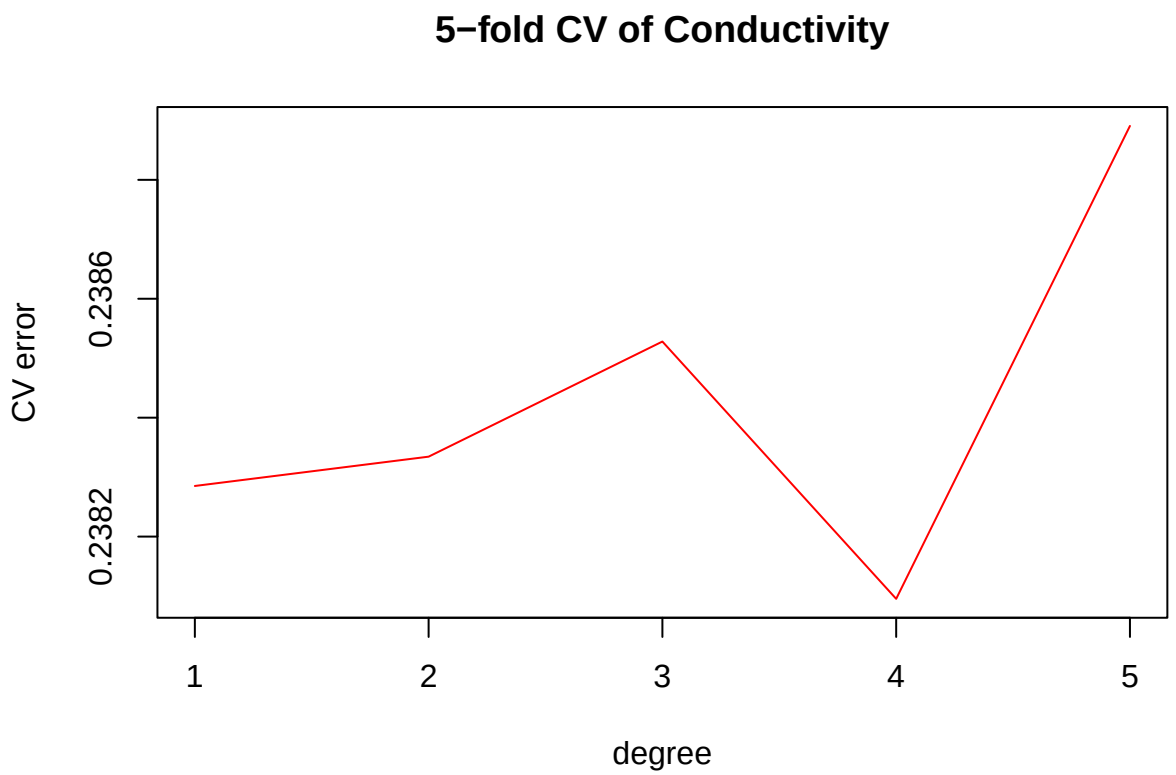
```
Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept)  -0.1838878  0.2304726  -0.798    0.425
Conductivity -0.0006160  0.0005314  -1.159    0.246

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 3068.5  on 2293  degrees of freedom
Residual deviance: 3067.2  on 2292  degrees of freedom
AIC: 3071.2

Number of Fisher Scoring iterations: 4
```

In this case $Pr(>|z|) > 0.05$ for β_1, then we cannot reject the hypothesis that $\beta_1 = 0$. This means that there is no evidence that the potability depends **only** on that variable.



There is no significant change in the CV error. All of this means that any polynomial until degree 5 does not affect significantly the result.

```
Confusion matrix for training set of Conductivity
glm.pred2.L.Co Not Potable Potable  Sum
   Not Potable        1399     895 2294
   Sum                1399     895 2294
Accuracy: 60.98518
```

```
Err: 39.01482

Confusion matrix for test set of Conductivity
glm.pred2.T.Co Not Potable Potable Sum
   Not Potable         599      383 982
   Sum                 599      383 982
Accuracy: 60.99796
Err: 39.00204
```

The result confirm our hypothesis: the model cannot predict correctly our result.

6.1.7 Organic_carbon

```

Call:
glm(formula = Potability ~ Organic_carbon, family = binomial,
    data = Water.L)

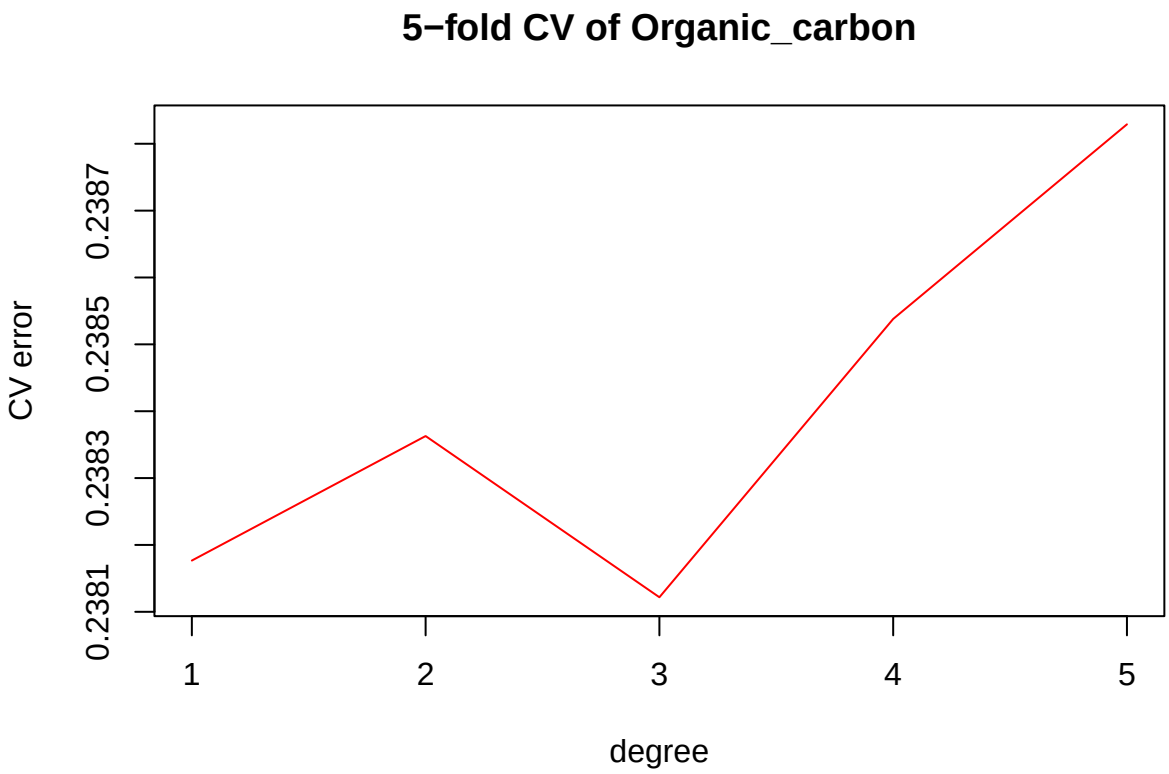
Coefficients:
               Estimate Std. Error z value Pr(>|z|)
(Intercept)    -0.17937    0.18542  -0.967    0.333
Organic_carbon -0.01881    0.01272  -1.479    0.139

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 3068.5  on 2293  degrees of freedom
Residual deviance: 3066.3  on 2292  degrees of freedom
AIC: 3070.3

Number of Fisher Scoring iterations: 4
```

In this case $Pr(>|z|) > 0.05$ for β_1, then we cannot reject the hypothesis that $\beta_1 = 0$. This means that there is no evidence that the potability depends **only** on that variable.



There is no significant change in the CV error. All of this means that any polynomial until degree 5 does not affect significantly the result.

```
Confusion matrix for training set of Organic_carbon
glm.pred2.L.0 Not Potable Potable  Sum
  Not Potable        1399     895 2294
  Sum                1399     895 2294
Accuracy: 60.98518
Err: 39.01482

Confusion matrix for test set of Organic_carbon
glm.pred2.T.0 Not Potable Potable Sum
  Not Potable         599     383 982
  Sum                 599     383 982
Accuracy: 60.99796
Err: 39.00204
```

The result confirm our hypothesis: the model cannot predict correctly our result.

6.1.8 Trihalomethanes

```

Call:
glm(formula = Potability ~ Trihalomethanes, family = binomial,
```

```
    data = Water.L)

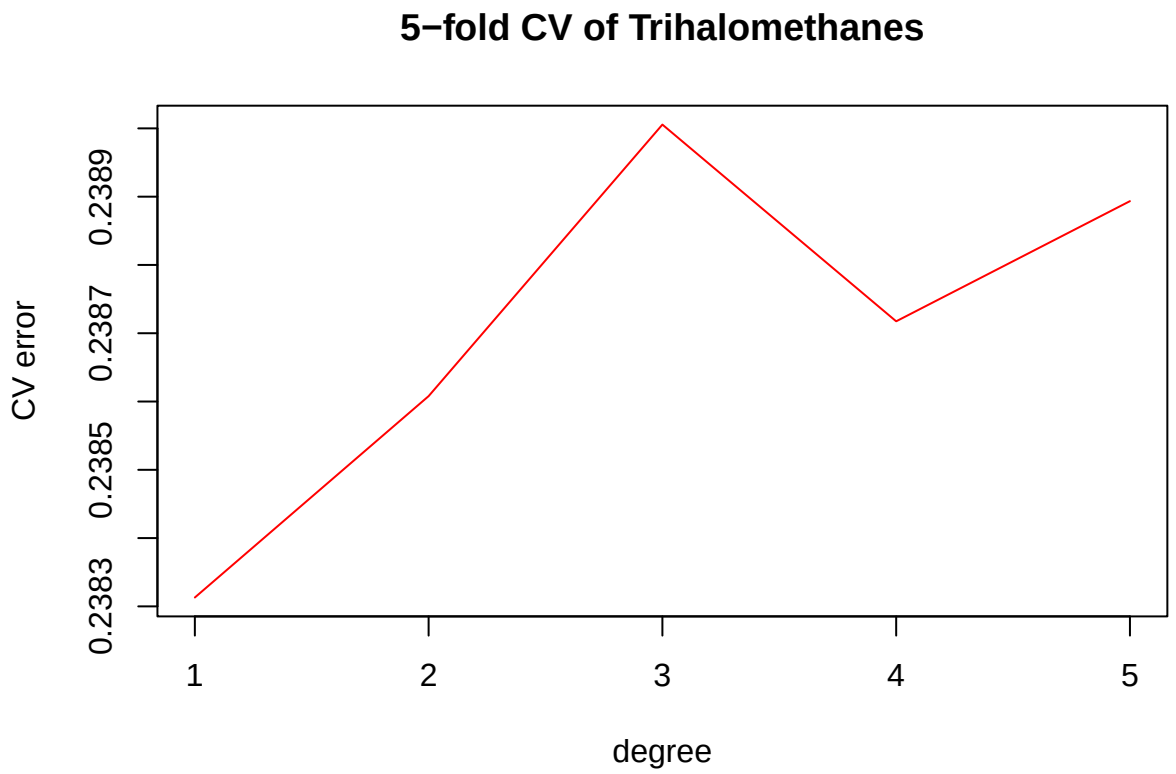
Coefficients:
                 Estimate Std. Error z value Pr(>|z|)
(Intercept)     -0.430154   0.187174  -2.298   0.0216 *
Trihalomethanes -0.000250   0.002756  -0.091   0.9277
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 3068.5  on 2293  degrees of freedom
Residual deviance: 3068.5  on 2292  degrees of freedom
AIC: 3072.5

Number of Fisher Scoring iterations: 4
```

In this case $Pr(>|z|) > 0.05$ for β_1, then we cannot reject the hypothesis that $\beta_1 = 0$. This means that there is no evidence that the potability depends **only** on that variable.



There is no significant change in the CV error. All of this means that any polynomial until degree 5 does not affect significantly the result.

```
Confusion matrix for training set of Trihalomethanes
glm.pred2.L.Tr Not Potable Potable  Sum
```

```
   Not Potable         1399      895 2294
   Sum                 1399      895 2294
Accuracy: 60.98518
Err: 39.01482
Confusion matrix for test set of Trihalomethanes
glm.pred2.T.Tr Not Potable Potable Sum
   Not Potable          599      383 982
   Sum                  599      383 982
Accuracy: 60.99796
Err: 39.00204
```

The result confirm our hypothesis: the model cannot predict correctly our result.

6.1.9 Turbidity

```

Call:
glm(formula = Potability ~ Turbidity, family = binomial, data = Water.L)

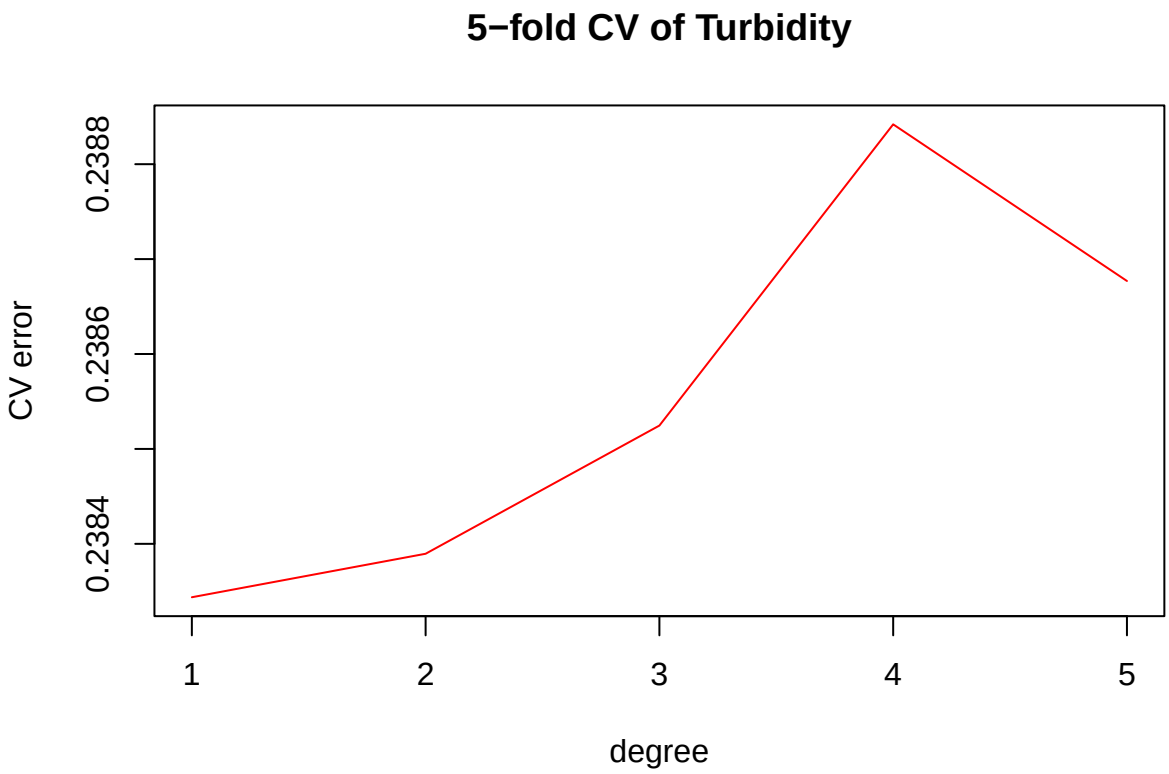
Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -0.56564    0.22082  -2.562   0.0104 *
Turbidity    0.03008    0.05475   0.549   0.5827
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 3068.5  on 2293  degrees of freedom
Residual deviance: 3068.2  on 2292  degrees of freedom
AIC: 3072.2

Number of Fisher Scoring iterations: 4
```

In this case $Pr(> |z|) > 0.05$ for β_1, then we cannot reject the hypothesis that $\beta_1 = 0$. This means that there is no evidence that the potability depends **only** on that variable.



There is no significant change in the CV error. All of this means that any polynomial until degree 5 does not affect significantly the result.

```
Confusion matrix for training set of Turbidity
glm.pred2.L.Tu Not Potable Potable  Sum
   Not Potable        1399      895 2294
   Sum                1399      895 2294
Accuracy: 60.98518
Err: 39.01482

Confusion matrix for test set of Turbidity
glm.pred2.T.Tu Not Potable Potable Sum
   Not Potable         599      383 982
   Sum                 599      383 982
Accuracy: 60.99796
Err: 39.00204
```

The result confirm our hypothesis: the model cannot predict correctly our result.

6.2 Multivariate Logistic regression

We want to estimate the best combination of predictors for our model(see Appendix B to how it's done). Also in this case we use the K-fold CV.

```
    Potability
Fold Not Potable Potable
  1           140      90
  2           140      90
  3           140      90
  4           140      90
  5           140      90
  6           140      89
  7           140      89
  8           140      89
  9           140      89
  10          139      89
------------------------------------------------------------------------
Best model:
Potability ~ ph + Hardness + Solids + Sulfate + Conductivity +
    Organic_carbon + Trihalomethanes
Number of predictors : 7
Predictors : ph, Hardness, Solids, Sulfate, Conductivity, Organic_carbon,
   Trihalomethanes
CV error: 0.388
------------------------------------------------------------------------
```

After choosing the best model on the training set, we will see the result on the test set.

```
Call:
glm(formula = best.model$formula, family = binomial, data = Water.L)

Coefficients:
                  Estimate Std. Error z value Pr(>|z|)
(Intercept)      3.558e-01  6.585e-01   0.540    0.589
ph               1.498e-02  2.953e-02   0.507    0.612
Hardness        -1.058e-03  1.315e-03  -0.805    0.421
Solids           9.357e-06  4.896e-06   1.911    0.056 .
Sulfate         -1.068e-03  1.201e-03  -0.889    0.374
Conductivity    -6.348e-04  5.335e-04  -1.190    0.234
Organic_carbon  -1.837e-02  1.276e-02  -1.440    0.150
Trihalomethanes -2.894e-04  2.763e-03  -0.105    0.917
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 3068.5  on 2293  degrees of freedom
Residual deviance: 3059.3  on 2286  degrees of freedom
AIC: 3075.3

Number of Fisher Scoring iterations: 4
```

From this result we can understand that the logistic regression cannot explain the data: using this model the combination of our predictors is not sufficient in predicting some patterns.

```
Confusion matrix for test set of Turbidity
predicted.m    Not Potable Potable Sum
  Not Potable          598     383 981
  Potable                1       0   1
  Sum                  599     383 982
Accuracy: 60.89613
Err: 39.10387
```

The Accuracy confirms our result: it's equal to the percentage of Not Potable water.

7 LDA

In this section we will evaluate the result of our LDA method.

```

pred_all       Not Potable Potable Sum
  Not Potable          598     382 980
  Potable                1       1   2
  Sum                  599     383 982
Accuracy:  60.99796
Error rat:  39.00204
```

The same exactly result: the model cannot predict our result.

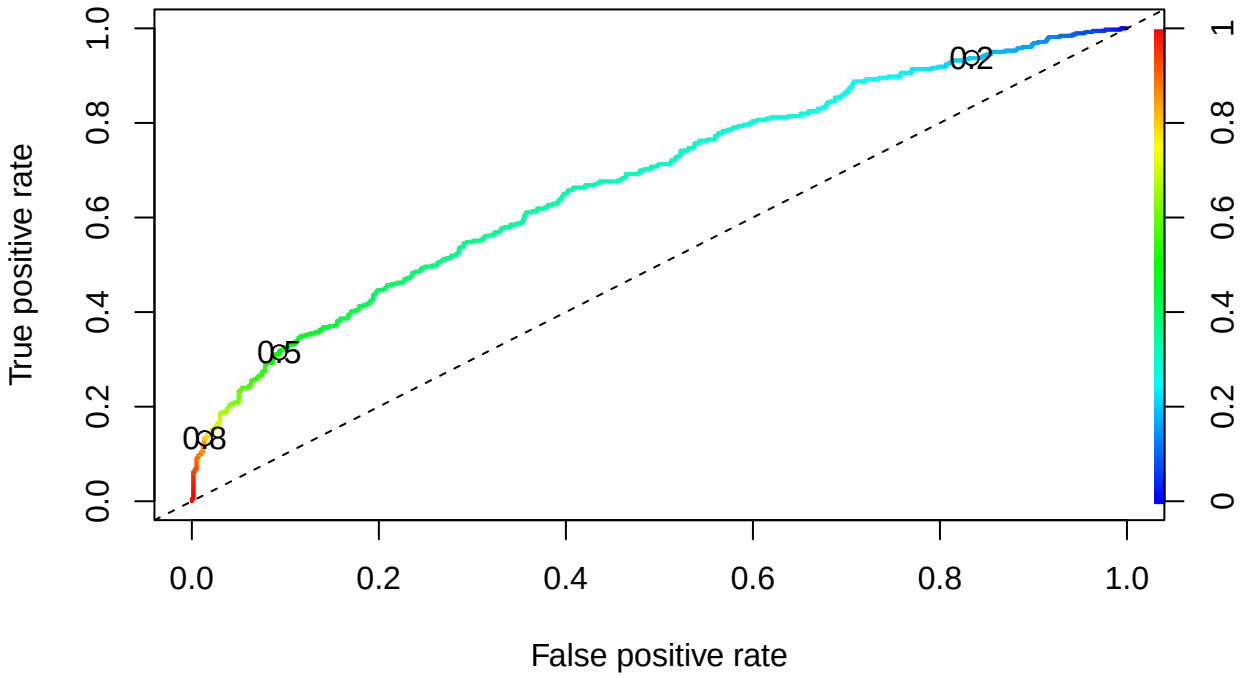
8 QDA

Let's try the QDA for our analysis.

```

pred_qda       Not Potable Potable Sum
  Not Potable          543     263 806
  Potable               56     120 176
  Sum                  599     383 982
Accuracy:  67.51527
Error rat:  32.48473
```

In this case the result is completely different. We can observe that only 56 Potable water samples are misclassified as not potable. We will analyze the ROC curve to evaluate the effect of changing the decision threshold trade-off.

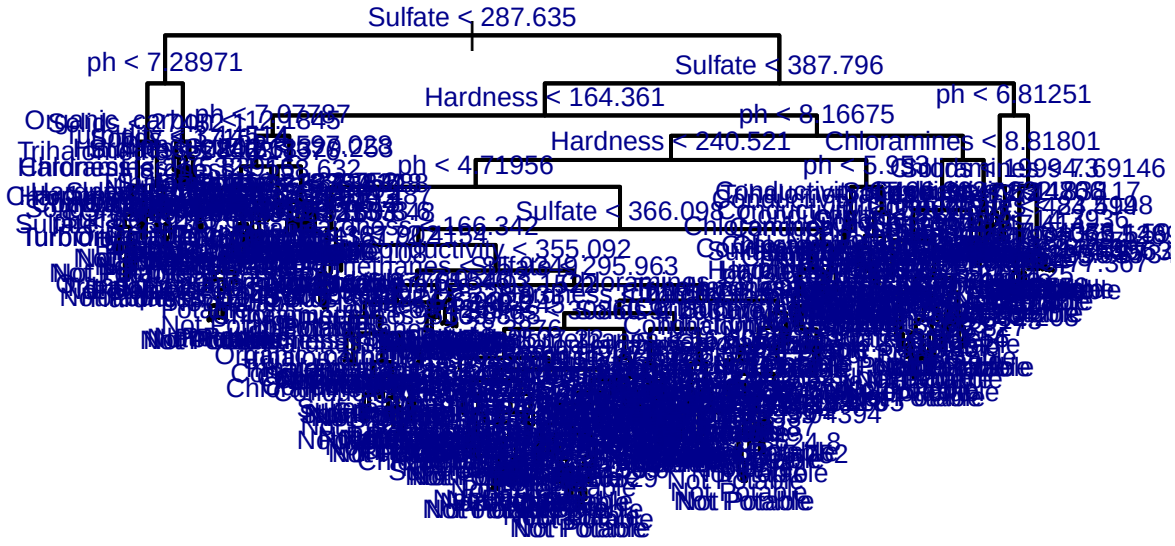



```
AUC:  0.6773474
```

Through our graph and AUC we can see a good prediction for our model.

9 Tree-based method

```
Classification tree:
tree(formula = Potability ~ ., data = Water.L, split = "gini")
Number of terminal nodes:  269
Residual mean deviance:  0.6328 = 1281 / 2025
Misclassification error rate: 0.1526 = 350 / 2294
```




```
tree.pred      Not Potable Potable Sum
  Not Potable          393     188 581
  Potable              206     195 401
  Sum                  599     383 982
Accuracy:  59.8778
Miss classification error rate:  40.1222
```

We used the Gini index to grow the initial tree, which resulted in 269 terminal nodes. Because this unpruned tree is overly complex and chaotic, it is necessary to perform cost-complexity pruning to create a more interpretable sub-tree.

```
Error in prune.tree(tree = structure(list(frame = structure(list(var = structure(1L, lev
```

```
Error: oggetto 'cv.water' non trovato
```

The algorithm for pruning doesn't work.

10 Bagging, random forest and boosting

We can try to use these other three methods to see what is the result.

10.1 Bagging

```
Call:
 randomForest(formula = Potability ~ ., data = Water.clear, mtry = 9,
    importance = TRUE, subset = train)
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 9

        OOB estimate of  error rate: 32.3%
Confusion matrix:
             Not Potable Potable class.error
Not Potable          857     147   0.1464143
Potable              382     252   0.6025237
```

```
             Actual
Predicted     Not Potable Potable  Sum
  Not Potable         833     384 1217
  Potable             161     260  421
  Sum                 994     644 1638
Accuracy:  66.72772
Miss classification error rate:  33.27228
```

10.2 Random forest

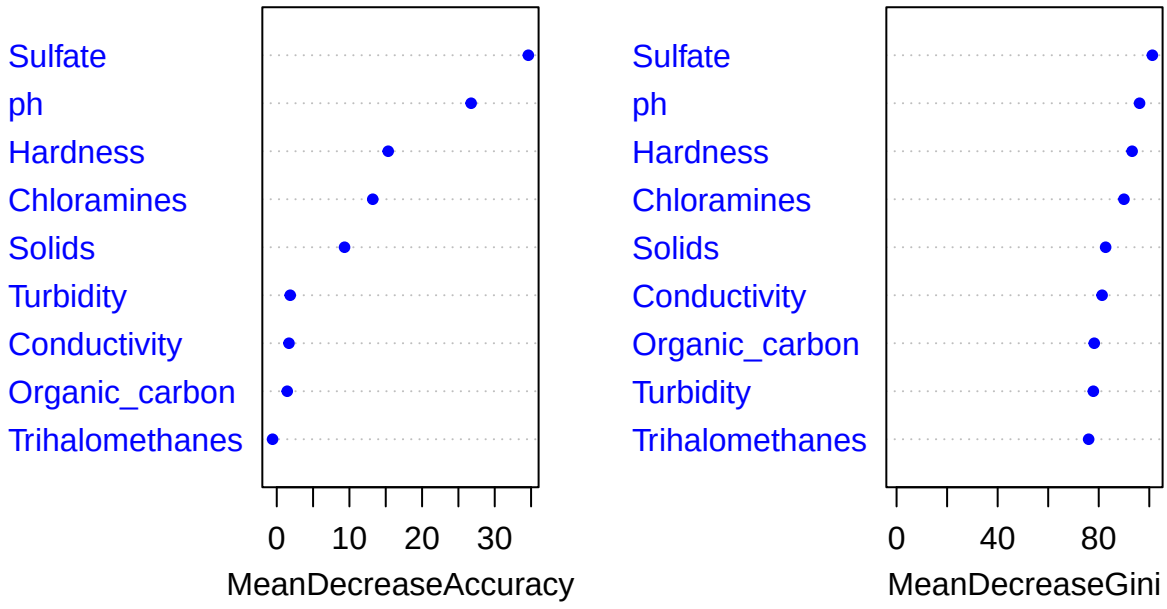
```
Call:
 randomForest(formula = Potability ~ ., data = Water.clear, mtry = 3,
    importance = TRUE, subset = train)
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 3

        OOB estimate of  error rate: 33.46%
Confusion matrix:
             Not Potable Potable class.error
Not Potable          864     140   0.1394422
Potable              408     226   0.6435331
```

```
             Actual
Predicted     Not Potable Potable  Sum
  Not Potable         842     413 1255
  Potable             152     231  383
  Sum                 994     644 1638
Accuracy:  65.50672
Miss classification error rate:  34.49328
```

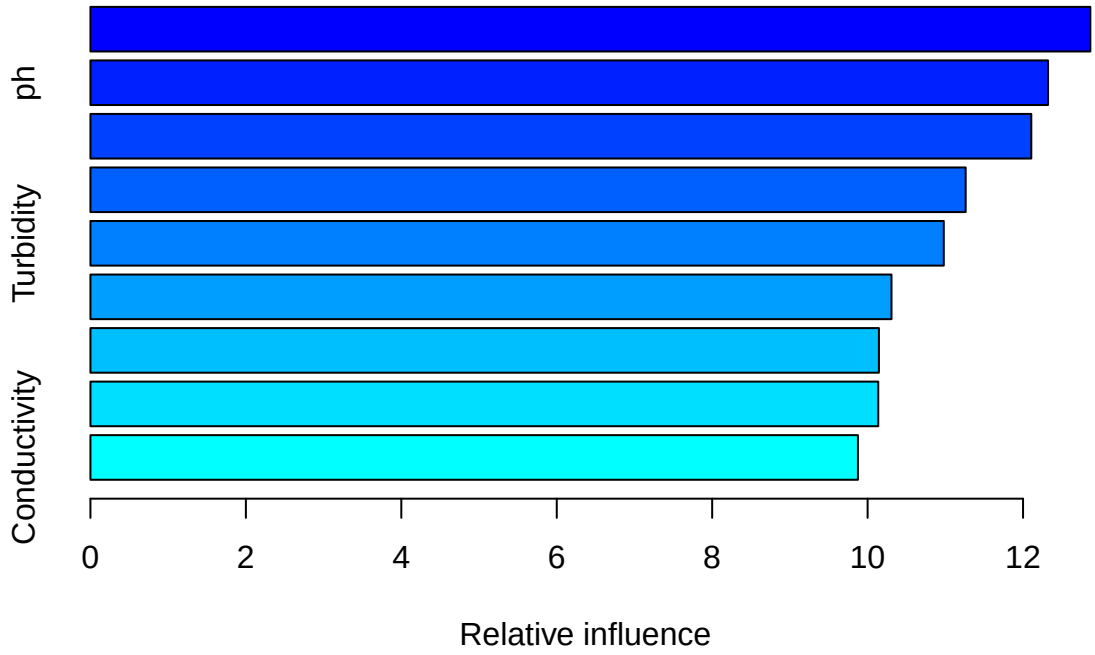
```
                 Not Potable     Potable MeanDecreaseAccuracy
   MeanDecreaseGini
ph               22.2000719 16.57639527             26.752214
   96.15115
Hardness         17.4644064  3.98580689             15.337856
   93.21108
Solids            4.4368317  9.33235833              9.329553
   82.73065
Chloramines       9.1903702 10.87337901             13.211889
   89.98093
Sulfate          24.3144395 26.04125809             34.627590
   101.22043
Conductivity      1.3041876  0.96420495              1.682410
   81.32921
Organic_carbon    2.4274652 -0.85471692              1.443956
   78.22265
Trihalomethanes  -0.7054919  0.06817838             -0.575317
   76.01280
Turbidity         1.7590783  0.91722465              1.854975
   77.86698
```

Water data



By analyzing the Mean Decrease Gini and Mean Decrease Accuracy plots, we can observe that Sulfate and pH are the most important variables for predicting potability

10.3 Boosting



	var	rel.inf
Sulfate	Sulfate	12.866003
ph	ph	12.321447
Hardness	Hardness	12.105989
Chloramines	Chloramines	11.261717
Turbidity	Turbidity	10.979882
Organic_carbon	Organic_carbon	10.307291
Solids	Solids	10.145352
Trihalomethanes	Trihalomethanes	10.136851
Conductivity	Conductivity	9.875467

```
             Actual
Predicted     Not Potable Potable  Sum
  Not Potable         842     414 1256
  Potable             152     230  382
  Sum                 994     644 1638
Accuracy:  65.44567
Miss classification error rate:  34.55433
```

11 Conclusion

What is happening to our analysis? Why do almost all the algorithms fail? The answer lies in the intrinsic structure of our dataset. If we closely examine the univariate analysis, we can observe that for potable and not potable water have the same distribution for each class. This means that Logistic Regression cannot detect a clear threshold to split the data based on potability.

Furthermore, observing the multivariate analysis (pairs plot), the data exhibits a clear ellipsoidal distribution. The true decision boundary between the two classes is an ellipse. This means that a linear model, such as Logistic Regression and LDA, cannot capture this geometry and so cannot separate the data between the two classes. Only QDA works because it is based on non-linear relations that can capture the shape of our data.

Why didn't pruning work? This is likely due to the nature of our data. The reduction in the classification error rate is so small that the algorithm fails, causing the model to collapse to a single root node. Conversely, the three ensemble methods succeed because they bypass this structural limitation. We can see that these methods significantly improve the overall results, although they ultimately achieve very similar accuracy among themselves.

This analysis demonstrates that non-linear and ensemble models are strictly required to properly classify this dataset.

12 Appendix A

Let's try a non linear logistic model. In our case only for demonstration we will use an equation associated to an ellipse with ph and Hardness as variables.

```
Call:
glm(formula = Potability ~ I(ph^2) + I(Hardness^2) + ph:Hardness +
    ph + Hardness, family = binomial, data = Water.L)

Coefficients:
                 Estimate Std. Error z value Pr(>|z|)
(Intercept)   -2.421e+00  1.641e+00  -1.475 0.140122
I(ph^2)       -4.917e-02  1.378e-02  -3.569 0.000359 ***
I(Hardness^2)  1.010e-04  2.638e-05   3.828 0.000129 ***
ph             1.207e+00  2.481e-01   4.866 1.14e-06 ***
Hardness      -2.202e-02  1.245e-02  -1.769 0.076850 .
ph:Hardness   -2.649e-03  7.948e-04  -3.333 0.000860 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 3068.5  on 2293  degrees of freedom
Residual deviance: 3025.6  on 2288  degrees of freedom
AIC: 3037.6

Number of Fisher Scoring iterations: 4
```

```
Confusion matrix for training set of Chloramines
non.linear.pred.L Not Potable Potable  Sum
      Not Potable        1353     836 2189
      Potable              46      59  105
      Sum                1399     895 2294
Accuracy: 61.55187
Err: 38.44813

Confusion matrix for test set of Chloramines
non.linear.pred.T Not Potable Potable Sum
      Not Potable         589     364 953
      Potable              10      19  29
      Sum                 599     383 982
Accuracy: 61.91446
Err: 38.08554
```

In this case we obtain a result, because we are using a non linear model that allows to recreate an appropriate shape for our data e separate them in a correct manner.

13 Appendix B

In this section I reposted all the most important codes inside our report.

13.1 Libraries

```
library(naniar)
library(visdat)
library(moments)
library(corrplot)
library(caret)
library(boot)
library(MASS)
library(ROCR)
library(tree)
```

13.2 NA values

```
Water.clear <- Water

# For cycle for each column
for(col in names(Water.clear)) {
  # Checking if the variable is numeric or not
  if(is.numeric(Water.clear[[col]])) {
    # Deriving the median, but removing the NA values from our calcolous
    median_col <- median(Water.clear[[col]], na.rm = TRUE)
    # Substituting the values
    Water.clear[[col]][is.na(Water.clear[[col]])] <- median_col
  }
  else{
    Water.clear[[col]] <- Water[[col]]
  }
}
```

13.3 Unaviarate analysis

```
# Taking only names of variables that do not correspond to Potability
cols <- setdiff(names(Water.clear), "Potability")


# Cycle for creating all boxplots
for (col in cols) {
  # Creating the row of graphs
```

```r
par(mfrow= c(1,3), oma = c(0, 0, 3, 0))

# Creating the boxplot
boxplot(
  Water.clear[[col]] ~ Water.clear$Potability,
  main = col,
  xlab = "Potability",
  ylab = "Values",
  col = c("#E06D53", "steelblue")
)
grid(nx = NA, ny = NULL, lty = 2, col = "gray80")
# Adding the box to the graph
box(which = "figure", col = "gray50", lwd = 1.5)

# Save the variable if Potable or Not Potable
NPot <- Water.clear[[col]][Water.clear$Potability == "Not Potable"]
Pot <- Water.clear[[col]][Water.clear$Potability == "Potable"]

# Creating the histogram for Not Potable
hist( NPot,
      breaks = "FD",
      main = paste("Histogram of ",col,"\n if Not Potable"),
      col = "#E06D53",
      border = "white",
      xlab = col,
      freq = TRUE)
grid(nx = NA, ny = NULL, lty = 2, col = "gray80")
box(which = "figure", col = "gray50", lwd = 1.5)

# Creating the histogram for Potable
hist( Pot,
      breaks = "FD",
      main = paste("Histogram of ",col,"\n if Potable"),
      col = "steelblue",
      border = "white",
      xlab = col,
      freq = TRUE)
grid(nx = NA, ny = NULL, lty = 2, col = "gray80")
box(which = "figure", col = "gray50", lwd = 1.5)

# Title for the line of graphs
mtext(col,
    side = 3,
    outer = TRUE,
```

```
      cex = 1.4,
      font = 2,
      line = 1)

  # Summary if Not Potable variable
  cat(paste("Summary of ", col , " if Not Potable\n"))
  cat("\n")
  print(summary(NPot))
  V<- var(NPot)
  cat("Variance : ",V,"\n")
  Sd <- sd(NPot)
  cat("Standard deviation : ",Sd,"\n")
  NPot.table <- table(NPot)
  mode <- names(NPot.table)[which.max(NPot.table)]
  cat("Mode : ",mode,"\n")
  K <- kurtosis(NPot)
  cat("Kurtosis : ",K,"\n")
  S <- skewness(NPot)
  cat("Skewness : ",S,"\n")
  cat("\n")
  cat("\n")

  # Summary if Potable variable
  cat(paste("Summary of ", col , " if Potable\n"))
  cat("\n")
  print(summary(Pot))
  V<- var(Pot)
  cat("Variance : ",V,"\n")
  Sd <- sd(Pot)
  cat("Standard deviation : ",Sd,"\n")
  Pot.table <- table(Pot)
  mode <- names(Pot.table)[which.max(Pot.table)]
  cat("Mode : ",mode,"\n")
  K <- kurtosis(Pot)
  cat("Kurtosis : ",K,"\n")
  S <- skewness(Pot)
  cat("Skewness : ",S,"\n")

}
```

13.4 Logistic regression

```
  fit.ph <- glm(Potability~ph, family = binomial, data = Water.L)
  print(summary(fit.ph, digits = 3))
```

```
cv.error.ph<-rep(0,5)
for(i in 1:5){
    poli.fit.ph <- glm(Potability~ poly(ph,i),family=binomial, data = Water.L)
    cv.error.ph[i]<-cv.glm(Water.L,poli.fit.ph,K=10)$delta[1]
  }
```

```
degree <- seq(1,5,1)
plot(degree,cv.error.ph, type="l", col="red",
       main="5-fold CV of ph",ylab = "CV error",)
```

```
glm.pred2.L.ph <- rep("Not Potable",nL)
glm.probs2.L.ph <- predict.glm(fit.ph,type="response")
glm.pred2.L.ph[glm.probs2.L.ph>.5]<-"Potable"
confMat2_L.ph<-addmargins(table(glm.pred2.L.ph,Water.L$Potability))
accuracy2_L.ph<-(confMat2_L.ph[1,1])/nL*100
Err2_L.ph<-100-accuracy2_L.ph

cat("Confusion matrix for training set of ph")
print(confMat2_L.ph)
cat("Accuracy:",accuracy2_L.ph,"\n")
cat("Err:",Err2_L.ph,"\n")
cat("\n")




glm.pred2.T.ph <- rep("Not Potable",nT)
glm.probs2.T.ph <- predict.glm(fit.ph,type="response",newdata = Water.T)
glm.pred2.T.ph[glm.probs2.T.ph>.5]<-"Potable"
confMat2_T.ph<-addmargins(table(glm.pred2.T.ph,Water.T$Potability))
accuracy2_T.ph<-(confMat2_T.ph[1,1])/nT*100
Err2_T.ph<-100-accuracy2_T.ph

cat("Confusion matrix for test set of ph")
print(confMat2_T.ph)
cat("Accuracy:",accuracy2_T.ph,"\n")
cat("Err:",Err2_T.ph,"\n")
```

In our model we modify the accuracy's formula, this because the original formula in our code will be written as follow:

```
accuracy2_T.ph<-(confMat2_T.ph[1,1]+confMat2_T.ph[2,2])/nT*100
```

It takes the diagonal elements, but the second element is the sum, not the count of predicted against true Potable. So I modified my formula like this:

```
accuracy2_T.ph<-(confMat2_T.ph[1,1])/nT*100
```

This is done for every variable because all the variables have the same problem.

13.5 Multivariate logistic regression

```
n.train <- nrow(Water.L)
p <- length(cols)
k <- 10

folds <- sample(rep(1:k, length = n.train))

# We want to create a manual k-fold partition of the data
# this because if we use the function cv.glm every time in the main for loop
# this will generate at each step a different k-fold and it is not technically
# the better method. We want that all the combination are trained on the same
# k fold data
idx_0 <- which(Water.L$Potability == "Not Potable")
idx_1 <- which(Water.L$Potability == "Potable")

folds[idx_0] <- sample(rep(1:k, length = length(idx_0)))
folds[idx_1] <- sample(rep(1:k, length = length(idx_1)))


print(table(Fold = folds, Potability = Water.L$Potability))

all.models <- list()
for (size in 1:p) {
  # Create a matrix where each column is a combination
  # of size "size" of predictors
  comb.mat <- combn(cols, size)

  # Cycle to create all the possible combination and save them
  # inside the list all models
  for (col.idx in 1:ncol(comb.mat)) {
    vars.subset <- comb.mat[, col.idx]
    all.models[[length(all.models) + 1]] <- list(
      size = size,
      vars = vars.subset,
      formula = as.formula(paste("Potability ~", paste(vars.subset, collapse = " + ")))
    )
  }
}
```

```r
# A vector that is the same size of the number of models
cv.err.rates <- numeric(length(all.models))


for (m in seq_along(all.models)) {
  # Taking the formula
  form.m <- all.models[[m]]$formula
  # Creating a vector of the size of k-fold in our case 10
  fold.errs <- numeric(k)

  # Cycle for the k-fold
  for (j in 1:k) {
    tr.fold  <- Water.L[folds != j, ]
    val.fold <- Water.L[folds == j, ]

    # Doing the fit on the selected formula and on the trian set of the k-fold
    fit <- glm(form.m, data = tr.fold, family = binomial)

    # Deriving the probabilities
    probs <- predict(fit, newdata = val.fold, type = "response")

    # Renaiming the prediction
    preds <- ifelse(probs > 0.5, "Potable", "Not Potable")

    # Defining the error rate for each k fold
    fold.errs[j] <- mean(preds != val.fold$Potability)
  }

  # Doing the mean of each k-fold to have a measure of the goodnes of that
  # possible cobination of predictors
  cv.err.rates[m] <- mean(fold.errs)
}

# Choosing the best index
best.idx <- which.min(cv.err.rates)
best.model <- all.models[[best.idx]]

cat(strrep("-", 70), "\n")
cat("Best model:\n")
print(best.model$formula)
cat("Number of predictors :", best.model$size, "\n")
cat("Predictors :", paste(best.model$vars, collapse = ", "), "\n")
cat("CV error:", round(cv.err.rates[best.idx], 4), "\n")
cat(strrep("-", 70), "\n")
```

13.6 LDA

```
lda_all <- lda(Potability ~ ., data = Water.L)
pred_all <- predict(lda_all, newdata = Water.T)$class
conf_LDA <- addmargins(table(pred_all,Water.T$Potability))
accuracy.LDA<-(conf_LDA[1,1]+conf_LDA[2,2])/nT*100
Err_LDA <- 100 - accuracy.LDA

print(conf_LDA)
cat("Accuracy: ",accuracy.LDA,"\n")
cat("Error rat: ",Err_LDA)
```

13.7 QDA

```
qda_all <- qda(Potability ~ ., data = Water.L)
pred_qda <- predict(qda_all, newdata = Water.T)$class
conf_qda <- addmargins(table(pred_qda,Water.T$Potability))
accuracy.qda<-(conf_qda[1,1]+conf_qda[2,2])/nT*100
Err_qda <- 100 - accuracy.qda

print(conf_qda)
cat("Accuracy: ",accuracy.qda,"\n")
cat("Error rat: ",Err_qda)
```

```
tree.water <- tree(Potability ~ ., data = Water.L)

summary(tree.water)

plot(tree.water, lwd = 2)
text(tree.water, pretty = 0, cex = 0.8, col = "darkblue")
```

13.8 Tree-method approach

```
tree.water <- tree(Potability ~ ., data = Water.L, split = "gini")

summary(tree.water)

plot(tree.water, lwd = 2)
text(tree.water, pretty = 0, cex = 0.8, col = "darkblue")
```

13.9 Bagging, rendom forest and boosting

```
train <- sample(1:nrow(Water.clear), nrow(Water.clear) / 2)
bag.water.R <- randomForest(Potability~ ., data = Water.clear,
                            subset = train,mtry = 3,importance = TRUE)
bag.water.R
```

```
Water.boost <- Water.clear
Water.boost$Potability <- ifelse(Water.boost$Potability == "Potable", 1, 0)

boost.water <- gbm(Potability~ ., data = Water.clear[train,],
                   distribution = "bernoulli", n.trees = 5000,
                   interaction.depth = 4)
summary(boost.water)
```